The Concise TypeScript Book

Simone Poggiali

The Concise TypeScript Book

The Concise TypeScript Book offre un aperçu complet et succinct des possibilités de TypeScript. Il propose des explications claires couvrant tous les aspects de la dernière version du langage, de son puissant système de types à ses fonctionnalités avancées.

Que vous soyez débutant ou développeur expérimenté, ce livre constitue une ressource précieuse pour approfondir votre compréhension et votre maîtrise de TypeScript.

Ce livre est entièrement gratuit et open source.

Je suis convaincu qu'une formation technique de qualité doit être accessible à tous. C'est pourquoi je laisse le livre en accès libre et le mets régulièrement à jour avec des améliorations et de nouveaux exemples.

Découvrez **The Concise TypeScript Book Plus Edition**.



Pour les lecteurs qui souhaitent aller au-delà de l'édition open source, **The Concise TypeScript Book Plus Edition: React and Real-World Patterns for TypeScript 7** propose du contenu supplémentaire et exclusif axé sur les applications pratiques.

La Plus Edition comprend :

- **Mise à jour pour TypeScript 7** — présentation des dernières fonctionnalités et améliorations du langage TypeScript 7.
- **TypeScript avec React** — conseils pratiques pour typer les composants, les props, les hooks, les événements, les enfants, les refs et les patterns React courants.

- **Recettes TypeScript pour des projets concrets** — exemples ciblés qui répondent aux problèmes pratiques rencontrés par les développeurs lors de la création et de la maintenance d'applications TypeScript.

En achetant la Plus Edition, vous soutenez aussi directement la poursuite du développement et de la maintenance du livre gratuit et open source.

La Plus Edition est disponible en anglais et en italien sur Amazon dans le monde entier. Découvrez la Plus Edition et achetez-la sur Amazon.

Soutenir le projet

Si le livre gratuit vous a aidé à corriger un bogue, à comprendre un concept difficile ou à progresser dans votre carrière, pensez à soutenir mon travail en versant le montant de votre choix, avec une contribution suggérée de **5 $**, ou en m'offrant un café.

Votre soutien m'aide à maintenir le contenu à jour et à l'enrichir de nouveaux exemples, d'explications plus claires et de conseils pratiques supplémentaires.





Traductions

Ce livre a été traduit dans plusieurs langues, notamment :

Anglais

Bulgare

Allemand

Français

Indonésien

Italien

Japonais

Coréen

Polonais

Portugais (Brésil)

Suédois

Turc

Vietnamese

Chinois

Espagnol

Thaïlandais

Russe

Arabe Tchèque

Téléchargements et site web

Vous pouvez également télécharger la version EPUB :

https://github.com/gibbok/typescript-book/tree/main/downloads

Une version en ligne est disponible à l'adresse suivante :

https://gibbok.github.io/typescript-book

Table des matières

- The Concise TypeScript Book
  - Soutenir le projet
  - Traductions
  - Téléchargements et site web
  - Table des matières
  - Introduction
  - À propos de l'auteur
  - Introduction à TypeScript
    - Qu'est-ce que TypeScript ?
    - Pourquoi TypeScript ?
    - TypeScript et JavaScript
    - Génération de code TypeScript
    - JavaScript moderne dès maintenant (Downleveling)
  - Bien démarrer avec TypeScript
    - Installation
    - Configuration
    - Fichier de configuration TypeScript
      - target
      - lib
      - strict
      - module

        - moduleResolution
        - esModuleInterop
        - jsx
        - skipLibCheck
        - files
        - include
        - exclude
    - importHelpers
    - Conseils pour migrer vers TypeScript
- Exploration du système de types
    - Le service de langage TypeScript
    - Typage structurel
    - Règles fondamentales de comparaison de TypeScript
    - Les types comme ensembles
    - Attribuer un type : déclarations et assertions de type
        - Déclaration de type
        - Assertion de type
        - Déclarations ambiantes
    - Vérification des propriétés et des propriétés excédentaires
    - Types faibles
    - Vérification stricte des littéraux d'objet (fraîcheur)
    - Inférence de type
    - Inférences plus avancées
    - Élargissement des types
    - Const
        - Modificateur Const sur les paramètres de type
        - Assertion Const
    - Annotation de type explicite
    - Réduction

        - Conditions
        - Lever une exception ou retourner une valeur
        - Union discriminée
        - Gardes de type définies par l'utilisateur
        - Réduction avec switch-true
    - Types primitifs
        - string
        - boolean
        - number
        - bigint
        - Symbol
        - null et undefined
        - Array
        - any
    - Annotations de type
    - Propriétés facultatives
    - Propriétés en lecture seule
    - Signatures d'index
    - Extension des types
    - Types littéraux
    - Inférence des littéraux
    - strictNullChecks
    - Énumérations
        - Énumérations numériques
        - Énumérations de chaînes
        - Énumérations constantes
        - Mappage inverse
        - Énumérations ambiantes
        - Membres calculés et constants
    - Réduction de type
        - Gardes de type typeof
        - Réduction par véracité

        - Réduction par égalité
        - Réduction avec l’opérateur in
        - Réduction avec instanceof
    - Affectations
    - Analyse du flux de contrôle
    - Prédicats de type
    - Unions discriminées
    - Le type never
    - Vérification de l’exhaustivité
    - Types d’objets
    - Type tuple (anonyme)
    - Type tuple nommé (étiqueté)
    - Tuple de longueur fixe
    - Type union
    - Types intersection
    - Indexation de type
    - Type à partir d’une valeur
    - Type à partir de la valeur de retour d’une fonction
    - Type à partir d’un module
    - Types mappés
    - Modificateurs de types mappés
    - Types conditionnels
    - Types conditionnels distributifs
    - Inférence de type avec infer dans les types conditionnels
    - Types conditionnels prédéfinis
    - Types union de littéraux de gabarit
    - Type any
    - Type unknown
    - Type void
    - Type never
    - Interface et type

        - Syntaxe courante
        - Types de base
        - Objets et interfaces
        - Types union et intersection
    - Types primitifs intégrés
    - Objets JS intégrés courants
    - Surcharges
    - Fusion et extension
    - Différences entre type et interface
    - Classe
        - Syntaxe courante d'une classe
        - Constructeur
        - Constructeurs privés et protégés
        - Modificateurs d'accès
        - Accesseurs get et set
        - Auto-accesseurs dans les classes
        - this
        - Propriétés de paramètres
        - Classes abstraites
        - Avec des génériques
        - Décorateurs
            - Décorateurs de classe
            - Décorateur de propriété
            - Décorateur de méthode
            - Décorateurs d'accesseurs et de mutateurs
            - Métadonnées des décorateurs
        - Héritage
        - Membres statiques
        - Initialisation des propriétés
        - Surcharge de méthodes
    - Génériques
        - Type générique

        - Classes génériques
        - Contraintes génériques
        - Réduction contextuelle des génériques
    - Types structurels effacés
    - Espaces de noms
    - Symboles
    - Directives à triple barre oblique
    - Manipulation des types
        - Création de types à partir de types
        - Types d’accès indexé
        - Types utilitaires
            - Awaited<T>
            - Partial<T>
            - Required<T>
            - Readonly<T>
            - Record<K, T>
            - Pick<T, K>
            - Omit<T, K>
            - Exclude<T, U>
            - Extract<T, U>
            - NonNullable<T>
            - Parameters<T>
            - ConstructorParameters<T>
            - ReturnType<T>
            - InstanceType<T>
            - ThisParameterType<T>
            - OmitThisParameter<T>
            - ThisType<T>
            - Uppercase<T>
            - Lowercase<T>
            - Capitalize<T>
            - Uncapitalize<T>

        - NoInfer<T>
    - Autres
        - Gestion des erreurs et des exceptions
        - Classes mixin
        - Fonctionnalités asynchrones du langage
        - Itérateurs et générateurs
        - Référence TsDocs JSDoc
        - @types
        - JSX
        - Modules ES6
        - Opérateur d’exponentiation ES7
        - L’instruction for-await-of
        - Métapropriété new.target
        - Expressions d’importation dynamique
        - “tsc –watch”
        - Opérateur d’assertion non-null
        - Déclarations avec valeurs par défaut
        - Chaînage optionnel
        - Opérateur de coalescence des valeurs nulles
        - Types littéraux de gabarit
        - Surcharge de fonctions
        - Types récursifs
        - Types conditionnels récursifs
        - Prise en charge des modules ECMAScript dans Node
        - Fonctions d’assertion
        - Types de tuples variadiques
        - Types d’objets enveloppeurs
        - Covariance et contravariance dans TypeScript
            - Annotations de variance facultatives pour les paramètres de type

- Signatures d’index avec motifs de chaînes de gabarit
- L’opérateur satisfies
- Importations et exportations de types uniquement
- Déclaration using et gestion explicite des ressources
  - Déclaration await using
- Attributs d’importation
- Vérification de la syntaxe des expressions régulières
- import defer

Introduction

Bienvenue dans The Concise TypeScript Book ! Ce guide vous apporte les connaissances essentielles et les compétences pratiques nécessaires pour développer efficacement avec TypeScript. Découvrez les concepts et les techniques clés qui permettent d’écrire du code propre et robuste. Que vous soyez débutant ou développeur expérimenté, ce livre constitue à la fois un guide complet et une référence pratique pour exploiter la puissance de TypeScript dans vos projets.

Ce livre couvre TypeScript 7.0.

À propos de l’auteur

Simone Poggiali est un Staff Engineer expérimenté, passionné par l’écriture de code de qualité professionnelle depuis les années 1990. Au cours de sa carrière internationale, il a contribué à de nombreux projets pour une grande variété de clients, des startups aux grandes organisations. Des entreprises

renommées telles que HelloFresh, Siemens, O2, Leroy Merlin et Snowplow ont bénéficié de son expertise et de son engagement.

Vous pouvez contacter Simone Poggiali sur les plateformes suivantes :

- LinkedIn : https://www.linkedin.com/in/simone-poggiali
- GitHub : https://github.com/gibbok
- Bluesky : https://bsky.app/profile/gibbok.bsky.social
- E-mail : gibbok.coding gmail.com

Liste complète des contributeurs : https://github.com/gibbok/typescript-book/graphs/contributors

Introduction à TypeScript

Qu’est-ce que TypeScript ?

TypeScript est un langage de programmation fortement typé qui repose sur JavaScript. Il a été initialement conçu par Anders Hejlsberg en 2012 et est actuellement développé et maintenu par Microsoft en tant que projet open source.

TypeScript est compilé en JavaScript et peut être exécuté dans n’importe quel environnement d’exécution JavaScript (par exemple, un navigateur ou Node.js sur un serveur).

Il prend en charge plusieurs paradigmes de programmation, tels que la programmation fonctionnelle, générique, impérative et orientée objet. Il s’agit d’un langage compilé (transpilé), converti en JavaScript avant son exécution.

Pourquoi TypeScript ?

TypeScript est un langage fortement typé qui aide à prévenir les erreurs de programmation courantes et à éviter certains types d'erreurs d'exécution avant que le programme ne soit exécuté.

Un langage fortement typé permet au développeur de spécifier diverses contraintes et divers comportements du programme dans les définitions de types de données, ce qui facilite la vérification de la justesse du logiciel et la prévention des défauts. Cela s'avère particulièrement utile dans les applications à grande échelle.

Voici quelques-uns des avantages de TypeScript :

- Typage statique, avec typage fort facultatif
- Inférence de type
- Accès aux fonctionnalités d'ES6 et d'ES7
- Compatibilité multiplateforme et multinavigateur
- Prise en charge par les outils avec IntelliSense

TypeScript et JavaScript

Le code TypeScript est écrit dans des fichiers `.ts` ou `.tsx`, tandis que les fichiers JavaScript utilisent les extensions `.js` ou `.jsx`.

Les fichiers portant l'extension `.tsx` ou `.jsx` peuvent contenir l'extension de syntaxe JavaScript JSX, utilisée dans React pour le développement d'interfaces utilisateur.

En matière de syntaxe, TypeScript est un sur-ensemble typé de JavaScript (ECMAScript 2015). Tout code JavaScript est un code TypeScript valide, mais l'inverse n'est pas toujours vrai.

Prenons par exemple une fonction dans un fichier JavaScript portant l'extension `.js`, comme celle-ci :

```
const sum = (a, b) => a + b;
```

La fonction peut être convertie et utilisée dans TypeScript en remplaçant l'extension du fichier par `.ts`. Cependant, si la même fonction est annotée avec des types TypeScript, elle ne peut être exécutée dans aucun environnement d'exécution JavaScript sans compilation. Le code TypeScript suivant produira une erreur de syntaxe s'il n'est pas compilé :

```
const sum = (a: number, b: number): number => a + b;
```

TypeScript a été conçu pour détecter les erreurs d'exécution potentielles au moment de la compilation, en permettant aux développeurs d'exprimer leur intention au moyen d'annotations de type. En outre, grâce à l'inférence de type, TypeScript peut également détecter certains problèmes même lorsqu'aucune annotation de type explicite n'est fournie. Par exemple, l'extrait de code suivant ne spécifie aucun type TypeScript :

```
const items = [{ x: 1 }, { x: 2 }];
const result = items.filter(item => item.y);
```

Dans ce cas, TypeScript détecte une erreur et indique :

```
Property 'y' does not exist on type '{ x: number; }'.
```

Le système de types de TypeScript est largement influencé par le comportement de JavaScript à l'exécution. Par exemple, l'opérateur d'addition (+), qui peut effectuer une concaténation de chaînes ou une addition numérique en JavaScript, est modélisé de la même manière dans TypeScript :

```
const result = '1' + 1; // Le résultat est de type string
```

L'équipe à l'origine de TypeScript a délibérément choisi de signaler comme erreurs les utilisations inhabituelles de JavaScript. Prenons par exemple le code JavaScript valide suivant :

```
const result = 1 + true; // En JavaScript, le résultat est égal à 2
```

Cependant, TypeScript génère une erreur :

```
Operator '+' cannot be applied to types 'number' and 'boolean'.
```

Cette erreur se produit parce que TypeScript applique strictement la compatibilité des types et identifie, dans ce cas, une opération non valide entre un nombre et un booléen.

Génération de code TypeScript

Le compilateur TypeScript assume deux responsabilités principales : rechercher les erreurs de type et compiler le code en JavaScript. Ces deux processus sont indépendants l'un de l'autre. Les types n'affectent pas l'exécution du code dans un environnement d'exécution JavaScript, car ils sont entièrement effacés lors de la compilation. TypeScript peut tout de même générer du JavaScript en présence d'erreurs de type. Voici un exemple de code TypeScript comportant une erreur de type :

```
const add = (a: number, b: number): number => a + b;
const result = add('x', 'y'); // Un argument de type 'string' n'est pas assignable à un paramètre de type 'number'.
```

Il peut néanmoins produire une sortie JavaScript exécutable :

```
'use strict';
const add = (a, b) => a + b;
const result = add('x', 'y'); // xy
```

Il n'est pas possible de vérifier les types TypeScript lors de l'exécution. Par exemple :

```
interface Animal {
    name: string;
}
interface Dog extends Animal {
    bark: () => void;
}
interface Cat extends Animal {
    meow: () => void;
}
const makeNoise = (animal: Animal) => {
    if (animal instanceof Dog) {
          // 'Dog' désigne uniquement un type, mais est utilisé
ici comme valeur.
        // ...
    }
};
```

Puisque les types sont effacés après la compilation, il n'existe aucun moyen d'exécuter ce code en JavaScript. Pour reconnaître les types lors de l'exécution, nous devons utiliser un autre mécanisme. TypeScript propose plusieurs options, dont l'une des plus courantes est l'« union étiquetée ». Par exemple :

```
interface Dog {
    kind: 'dog'; // Union discriminée
    bark: () => void;
}
interface Cat {
    kind: 'cat'; // Union discriminée
```

```
    meow: () => void;
}
type Animal = Dog | Cat;

const makeNoise = (animal: Animal) => {
    if (animal.kind === 'dog') {
        animal.bark();
    } else {
        animal.meow();
    }
};

const dog: Dog = {
    kind: 'dog',
    bark: () => console.log('bark'),
};
makeNoise(dog);
```

La propriété « kind » est une valeur qui peut être utilisée lors de l'exécution pour distinguer les objets en JavaScript.

Il est également possible qu'une valeur possède, lors de l'exécution, un type différent de celui indiqué dans la déclaration de type. Cela peut par exemple se produire si le développeur a mal interprété le type d'une API et l'a annoté de manière incorrecte.

TypeScript étant un sur-ensemble de JavaScript, le mot-clé « class » peut être utilisé comme type et comme valeur lors de l'exécution.

```
class Animal {
    constructor(public name: string) {}
}
class Dog extends Animal {
    constructor(
        public name: string,
```

```
        public bark: () => void
    ) {
        super(name);
    }
}
class Cat extends Animal {
    constructor(
        public name: string,
        public meow: () => void
    ) {
        super(name);
    }
}
type Mammal = Dog | Cat;

const makeNoise = (mammal: Mammal) => {
    if (mammal instanceof Dog) {
        mammal.bark();
    } else {
        mammal.meow();
    }
};

const dog = new Dog('Fido', () => console.log('bark'));
makeNoise(dog);
```

En JavaScript, une « class » possède une propriété « prototype », et l'opérateur « instanceof » permet de vérifier si la propriété prototype d'un constructeur apparaît quelque part dans la chaîne de prototypes d'un objet.

TypeScript n'a aucun effet sur les performances lors de l'exécution, car tous les types sont effacés. Cependant, TypeScript introduit un certain surcoût lors de la compilation.

JavaScript moderne dès maintenant (Downleveling)

TypeScript peut compiler du code vers n'importe quelle version publiée de JavaScript depuis ECMAScript 3 (1999). Cela signifie que TypeScript peut transpiler du code utilisant les dernières fonctionnalités de JavaScript vers des versions antérieures, un processus appelé Downleveling. Il est ainsi possible d'utiliser du JavaScript moderne tout en conservant une compatibilité maximale avec les environnements d'exécution plus anciens.

Il est important de noter que, lors de la transpilation vers une ancienne version de JavaScript, TypeScript peut générer du code susceptible d'entraîner une surcharge de performances par rapport aux implémentations natives.

Voici quelques-unes des fonctionnalités JavaScript modernes qui peuvent être utilisées dans TypeScript :

- Les modules ECMAScript au lieu des callbacks « define » de style AMD ou des instructions « require » de CommonJS.
- Les classes au lieu des prototypes.
- La déclaration de variables avec « let » ou « const » au lieu de « var ».
- La boucle « for-of » ou « .forEach » au lieu de la boucle « for » traditionnelle.
- Les fonctions fléchées au lieu des expressions de fonction.
- L'affectation par déstructuration.
- Les noms abrégés de propriétés et de méthodes ainsi que les noms de propriétés calculés.
- Les paramètres de fonction par défaut.

En exploitant ces fonctionnalités JavaScript modernes, les développeurs peuvent écrire du code TypeScript plus expressif et plus concis.

Bien démarrer avec TypeScript

Installation

Visual Studio Code offre une excellente prise en charge du langage TypeScript, mais n'inclut pas le compilateur TypeScript. Pour installer ce dernier, vous pouvez utiliser un gestionnaire de paquets comme npm ou yarn :

```
npm install typescript --save-dev
```

ou

```
yarn add typescript --dev
```

Veillez à versionner le fichier de verrouillage généré afin que chaque membre de l'équipe utilise la même version de TypeScript.

Pour exécuter le compilateur TypeScript, vous pouvez utiliser les commandes suivantes :

```
npx tsc
```

ou

```
yarn tsc
```

Il est recommandé d'installer TypeScript au niveau du projet plutôt que globalement, car cela rend le processus de génération plus prévisible. Cependant, pour des utilisations ponctuelles, vous pouvez employer la commande suivante :

```
npx tsc
```

ou l'installer globalement :

```
npm install -g typescript
```

Si vous utilisez Microsoft Visual Studio, vous pouvez obtenir TypeScript sous forme de paquet NuGet pour vos projets MSBuild. Dans la console du gestionnaire de paquets NuGet, exécutez la commande suivante :

```
Install-Package Microsoft.TypeScript.MSBuild
```

Lors de l'installation de TypeScript, deux exécutables sont installés : « tsc », le compilateur TypeScript, et « tsserver », le serveur TypeScript autonome. Ce serveur autonome contient le compilateur et les services de langage que les éditeurs et les IDE peuvent utiliser pour proposer une complétion de code intelligente.

Par ailleurs, plusieurs transpileurs compatibles avec TypeScript sont disponibles, comme Babel (au moyen d'un plugin) ou swc. Ces transpileurs peuvent être utilisés pour convertir du code TypeScript vers d'autres langages ou versions cibles.

TypeScript 7.0 a été réécrit en Go sous la forme d'une implémentation native du compilateur et du service de langage. Il utilise le multithreading à mémoire partagée et d'autres optimisations pour accélérer les générations complètes et les fonctionnalités des éditeurs, réduisant ainsi le temps de réponse pendant le développement.

Certaines fonctionnalités de TypeScript 7.0 liées aux performances peuvent être ajustées. La vérification des types peut s'exécuter dans des workers parallèles avec `--checkers` ; un plus grand nombre de workers peut accélérer les grands

projets, mais consomme davantage de mémoire. Le mode `--watch` réécrit améliore la surveillance multiplateforme des fichiers. TypeScript 7.0 ne comprend pas encore d'API du compilateur (en juillet 2026), de sorte que les outils qui ont toujours besoin de l'API TypeScript 6.0 peuvent s'exécuter en parallèle avec TypeScript 7.0 en utilisant `@typescript/typescript6` ou des alias npm.

Configuration

TypeScript peut être configuré à l'aide des options de la CLI tsc ou d'un fichier de configuration dédié, nommé tsconfig.json et placé à la racine du projet.

Pour générer un fichier tsconfig.json prérempli avec les paramètres recommandés, vous pouvez utiliser la commande suivante :

```
tsc --init
```

Lorsque vous exécutez localement la commande `tsc`, TypeScript compile le code en utilisant la configuration définie dans le fichier tsconfig.json le plus proche.

Voici quelques exemples de commandes CLI exécutées avec les paramètres par défaut :

```
tsc main.ts // Compiler un fichier précis (main.ts) en JavaScript
tsc src/*.ts // Compiler tous les fichiers .ts du dossier 'src' en JavaScript
tsc app.ts util.ts --outfile index.js // Compiler deux fichiers TypeScript (app.ts et util.ts) en un seul fichier JavaScript (index.js)
```

Fichier de configuration TypeScript

Un fichier tsconfig.json sert à configurer le compilateur TypeScript (tsc). Il est généralement ajouté à la racine du projet, avec le fichier `package.json`.

Remarques :

- tsconfig.json accepte les commentaires même s'il est au format json.
- Il est conseillé d'utiliser ce fichier de configuration plutôt que les options de ligne de commande.

Le lien suivant fournit la documentation complète ainsi que son schéma :

https://www.typescriptlang.org/tsconfig

https://www.typescriptlang.org/tsconfig/

Voici une liste des configurations courantes et utiles :

target

La propriété « target » permet de spécifier la version d'ECMAScript vers laquelle votre code TypeScript doit être émis/compilé. Pour les navigateurs modernes, ES6 constitue une bonne option. Remarque : la prise en charge d'ES5 a été dépréciée dans TypeScript 6.0 et n'est plus assurée dans TypeScript 7.0.

lib

La propriété « lib » permet de spécifier les fichiers de bibliothèque à inclure lors de la compilation. TypeScript inclut automatiquement les API correspondant aux fonctionnalités spécifiées dans la propriété « target », mais il est possible d'omettre certaines bibliothèques ou d'en sélectionner en fonction de besoins particuliers. Par exemple, si vous travaillez sur un projet côté serveur, vous pouvez exclure la bibliothèque « DOM », qui n'est utile que dans un environnement de navigateur.

strict

L'option « strict » améliore la sécurité du typage en activant des vérifications plus strictes. Elle est activée par défaut à partir de TypeScript 6.0 ; dans le cas contraire, vous devez explicitement lui attribuer la valeur true dans votre tsconfig.json. L'activation de « strict » permet à TypeScript de :

- Émettre du code utilisant « use strict » pour chaque fichier source.
- Prendre en compte « null » et « undefined » lors de la vérification des types.
- Interdire l'utilisation du type « any » en l'absence d'annotations de type.
- Signaler une erreur lors de l'utilisation de l'expression « this », qui impliquerait autrement le type « any ».

module

La propriété « module » définit le système de modules pris en charge pour le programme compilé. Lors de l'exécution, un

chargeur de modules est utilisé pour localiser et exécuter les dépendances selon le système de modules spécifié.

Les chargeurs de modules les plus couramment utilisés en JavaScript sont Node.js CommonJS pour les applications côté serveur et RequireJS pour les modules AMD dans les applications Web exécutées dans un navigateur. TypeScript peut émettre du code pour différents systèmes de modules, notamment UMD, System, ESNext, ES2015/ES6 et ES2020. Le système de modules doit être choisi en fonction de l'environnement cible et du mécanisme de chargement de modules disponible dans cet environnement.

Remarque : la prise en charge des anciens systèmes de modules (AMD, UMD, SystemJS) a été dépréciée dans TypeScript 6.0 et n'est plus assurée dans TypeScript 7.0.

moduleResolution

La propriété « moduleResolution » spécifie la stratégie de résolution des modules. Utilisez « nodenext » ou « bundler » pour le code TypeScript moderne. La stratégie « classic » n'est utilisée que pour les anciennes versions de TypeScript (antérieures à 1.6).

esModuleInterop

La propriété « esModuleInterop » autorise les imports par défaut depuis des modules CommonJS qui n'ont pas exporté de valeur à l'aide de la propriété « default » ; cette propriété fournit une couche de compatibilité dans le code JavaScript émis. Après avoir activé cette option, nous pouvons utiliser

`import MyLibrary from "my-library"` au lieu de `import * as MyLibrary from "my-library"`.

À l'origine, « esModuleInterop » était facultative afin d'éviter les changements incompatibles, mais elle constitue depuis longtemps la valeur par défaut recommandée. Sa désactivation peut provoquer de subtils problèmes d'exécution lors de l'utilisation de CommonJS avec ESM. Remarque : à partir de TypeScript 6.0, ce comportement d'interopérabilité plus sûr est toujours activé.

Dans TypeScript 6.0, certaines anciennes options de configuration et formes syntaxiques ont été dépréciées ou sont désormais traitées comme du comportement hérité. Dans TypeScript 7.0, elles produisent des erreurs bloquantes ou n'ont aucun effet.

Les éléments dépréciés qui produisent désormais des erreurs bloquantes et n'ont aucun effet sont :

- `target: es5`
- `downlevelIteration`
- `moduleResolution: node/node10`
- `module: amd/umd/systemjs/none`
- `baseUrl`
- `moduleResolution: classic`
- la désactivation de `esModuleInterop` ou de `allowSyntheticDefaultImports`
- la désactivation de `alwaysStrict`
- le mot-clé `module` dans les déclarations d'espace de noms
- `asserts` dans les imports
- `/// <reference no-default-lib />` avec `skipDefaultLibCheck`

- les chemins de fichiers de la CLI avec un `tsconfig.json` local, sauf si `--ignoreConfig` est utilisé

jsx

La propriété « jsx » s'applique uniquement aux fichiers .tsx utilisés dans ReactJS et contrôle la manière dont les constructions JSX sont compilées en JavaScript. Une option courante est « preserve », qui compile vers un fichier .jsx en conservant le JSX inchangé afin qu'il puisse être transmis à différents outils tels que Babel pour d'autres transformations.

skipLibCheck

La propriété « skipLibCheck » empêche TypeScript de vérifier les types de l'intégralité des paquets tiers importés. Cette propriété réduit le temps de compilation d'un projet. TypeScript continue toutefois de vérifier votre code par rapport aux définitions de type fournies par ces paquets.

files

La propriété « files » indique au compilateur une liste de fichiers qui doivent toujours être inclus dans le programme.

include

La propriété « include » indique au compilateur une liste de fichiers que nous souhaitons inclure. Cette propriété autorise des motifs similaires aux globs, tels que « *_ » pour tout sous-répertoire, « _ » pour tout nom de fichier et « ? » pour les caractères facultatifs.

exclude

La propriété « exclude » indique au compilateur une liste de fichiers qui ne doivent pas être inclus dans la compilation. Il peut notamment s'agir de fichiers tels que ceux de « node_modules » ou de fichiers de test. Remarque : tsconfig.json autorise les commentaires.

importHelpers

TypeScript utilise du code d'assistance lors de la génération de code pour certaines fonctionnalités JavaScript avancées ou rétrocompilées. Par défaut, ces fonctions d'assistance sont dupliquées dans les fichiers qui les utilisent. L'option `importHelpers` importe plutôt ces fonctions d'assistance depuis le module `tslib`, ce qui rend le code JavaScript produit plus efficace.

Conseils pour migrer vers TypeScript

Pour les projets de grande taille, il est recommandé d'adopter une transition progressive au cours de laquelle le code TypeScript et le code JavaScript coexistent initialement. Seuls les petits projets peuvent être migrés vers TypeScript en une seule fois.

La première étape de cette transition consiste à introduire TypeScript dans la chaîne de compilation. Cela peut être fait à l'aide de l'option de compilation « allowJs », qui permet aux fichiers .ts et .tsx de coexister avec les fichiers JavaScript existants. Comme TypeScript se rabat sur le type « any » pour une variable lorsqu'il ne peut pas en déduire le type à partir

des fichiers JavaScript, il est recommandé de désactiver « noImplicitAny » dans les options de compilation au début de la migration.

La deuxième étape consiste à s'assurer que vos tests JavaScript fonctionnent avec les fichiers TypeScript afin de pouvoir exécuter les tests à mesure que vous convertissez chaque module. Si vous utilisez Jest, envisagez d'utiliser `ts-jest`, qui permet de tester des projets TypeScript avec Jest.

La troisième étape consiste à inclure dans votre projet les déclarations de type des bibliothèques tierces. Ces déclarations peuvent être fournies avec les bibliothèques ou être disponibles sur DefinitelyTyped. Vous pouvez les rechercher à l'adresse https://www.typescriptlang.org/dt/search et les installer à l'aide de :

```
npm install --save-dev @types/package-name
```

ou

```
yarn add --dev @types/package-name
```

La quatrième étape consiste à migrer module par module selon une approche ascendante, en suivant votre graphe de dépendances et en commençant par ses feuilles. L'idée est de commencer par convertir les modules qui ne dépendent pas d'autres modules. Pour visualiser les graphes de dépendances, vous pouvez utiliser l'outil « madge ».

Les fonctions utilitaires et le code lié à des API ou à des spécifications externes constituent de bons candidats pour ces premières conversions. Il est possible de générer automatiquement des définitions de type TypeScript à partir de

contrats Swagger, de schémas GraphQL ou de schémas JSON afin de les inclure dans votre projet.

Lorsqu'aucune spécification ni aucun schéma officiel ne sont disponibles, vous pouvez générer des types à partir de données brutes, telles que du JSON renvoyé par un serveur. Il est toutefois recommandé de générer les types à partir de spécifications plutôt que de données afin d'éviter d'omettre des cas limites.

Pendant la migration, évitez de refactoriser le code et concentrez-vous uniquement sur l'ajout de types à vos modules.

La cinquième étape consiste à activer « noImplicitAny », qui impose que tous les types soient connus et définis, offrant ainsi une meilleure expérience TypeScript pour votre projet.

Pendant la migration, vous pouvez utiliser la directive `@ts-check`, qui active la vérification des types TypeScript dans un fichier JavaScript. Cette directive fournit une forme souple de vérification des types et peut être utilisée dans un premier temps pour repérer les problèmes dans les fichiers JavaScript. Lorsque `@ts-check` est inclus dans un fichier, TypeScript tente de déduire les définitions à l'aide de commentaires de style JSDoc. Toutefois, envisagez d'utiliser les annotations JSDoc uniquement à un stade très précoce de la migration.

Envisagez de conserver la valeur par défaut de `noEmitOnError` dans votre tsconfig.json, à savoir false. Cela vous permet de produire le code source JavaScript même si des erreurs sont signalées.

Exploration du système de types

Le service de langage TypeScript

Le service de langage TypeScript, également appelé tsserver, offre diverses fonctionnalités telles que le signalement des erreurs, les diagnostics, la compilation lors de l'enregistrement, le renommage, l'accès à la définition, les listes de complétion, l'aide sur les signatures et bien plus encore. Il est principalement utilisé par les environnements de développement intégrés (IDE) pour assurer la prise en charge d'IntelliSense. Il s'intègre parfaitement à Visual Studio Code et est utilisé par des outils tels que Conquer of Completion (Coc).

Les développeurs peuvent exploiter une API dédiée et créer leurs propres plugins personnalisés pour le service de langage afin d'améliorer l'expérience d'édition de TypeScript. Cela peut être particulièrement utile pour mettre en œuvre des fonctionnalités spécifiques de linting ou activer l'autocomplétion pour un langage de gabarits personnalisé.

« typescript-styled-plugin » est un exemple concret de plugin personnalisé : il signale les erreurs de syntaxe et prend en charge IntelliSense pour les propriétés CSS dans les composants stylisés.

Pour obtenir davantage d'informations et consulter des guides de démarrage rapide, vous pouvez vous référer au wiki officiel de TypeScript sur GitHub : https://github.com/microsoft/TypeScript/wiki/

Typage structurel

TypeScript repose sur un système de types structurel. Cela signifie que la compatibilité et l'équivalence des types sont

déterminées par leur structure ou leur définition réelle, plutôt que par leur nom ou leur lieu de déclaration, comme c'est le cas dans les systèmes de types nominaux tels que ceux de C# ou C.

Le système de types structurel de TypeScript a été conçu d'après le fonctionnement du typage canard dynamique de JavaScript lors de l'exécution.

L'exemple suivant est du code TypeScript valide. Comme vous pouvez le constater, « X » et « Y » possèdent le même membre « a », bien qu'ils aient des noms de déclaration distincts. Les types sont déterminés par leurs structures et, dans ce cas, puisque celles-ci sont identiques, ils sont compatibles et valides.

```
type X = {
    a: string;
};
type Y = {
    a: string;
};
const x: X = { a: 'a' };
const y: Y = x; // Valide
```

Règles fondamentales de comparaison de TypeScript

Le processus de comparaison de TypeScript est récursif et s'applique aux types imbriqués à n'importe quel niveau.

Un type « X » est compatible avec « Y » si « Y » possède au moins les mêmes membres que « X ».

```
type X = {
    a: string;
};
```

```
const y = { a: 'A', b: 'B' }; // Valide, car il possède au
moins les mêmes membres que X
const r: X = y;
```

Les paramètres de fonction sont comparés selon leur type, et non selon leur nom :

```
type X = (a: number) => void;
type Y = (a: number) => void;
let x: X = (j: number) => undefined;
let y: Y = (k: number) => undefined;
y = x; // Valide
x = y; // Valide
```

Les types de retour des fonctions doivent être identiques :

```
type X = (a: number) => undefined;
type Y = (a: number) => number;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => 1;
y = x; // Invalide
x = y; // Invalide
```

Le type de retour d'une fonction source doit être un sous-type du type de retour d'une fonction cible :

```
let x = () => ({ a: 'A' });
let y = () => ({ a: 'A', b: 'B' });
x = y; // Valide
y = x; // Invalide : le membre b est manquant
```

L'omission de paramètres de fonction est autorisée, car il s'agit d'une pratique courante en JavaScript, par exemple avec « Array.prototype.map() » :

```
[1, 2, 3].map((element, _index, _array) => element + 'x');
```

Par conséquent, les déclarations de type suivantes sont tout à fait valides :

```
type X = (a: number) => undefined;
type Y = (a: number, b: number) => undefined;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => undefined; // Paramètre b manquant
y = x; // Valide
```

Tous les paramètres facultatifs supplémentaires du type source sont valides :

```
type X = (a: number, b?: number, c?: number) => undefined;
type Y = (a: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Valide
x = y; //Valide
```

Tous les paramètres facultatifs du type cible sans paramètres correspondants dans le type source sont valides et ne constituent pas une erreur :

```
type X = (a: number) => undefined;
type Y = (a: number, b?: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Valide
x = y; // Valide
```

Le paramètre rest est traité comme une série infinie de paramètres facultatifs :

```
type X = (a: number, ...rest: number[]) => undefined;
let x: X = a => undefined; //valide
```

Les fonctions avec des surcharges sont valides si la signature de surcharge est compatible avec leur signature d'implémentation :

```
function x(a: string): void;
function x(a: string, b: number): void;
function x(a: string, b?: number): void {
    console.log(a, b);
}
x('a'); // Valide
x('a', 1); // Valide

function y(a: string): void; // Invalide, incompatible avec la signature d'implémentation
function y(a: string, b: number): void;
function y(a: string, b: number): void {
    console.log(a, b);
}
y('a');
y('a', 1);
```

La comparaison des paramètres de fonction réussit si les paramètres source et cible peuvent être affectés à des supertypes ou des sous-types (bivariance).

```
// Supertype
class X {
    a: string;
    constructor(value: string) {
        this.a = value;
    }
}
// Sous-type
class Y extends X {}
// Sous-type
class Z extends X {}

type GetA = (x: X) => string;
```

```
const getA: GetA = x => x.a;

// La bivariance accepte les supertypes
console.log(getA(new X('x'))); // Valide
console.log(getA(new Y('Y'))); // Valide
console.log(getA(new Z('z'))); // Valide
```

Les énumérations sont comparables et compatibles avec les nombres, et inversement, mais la comparaison de valeurs d'énumération provenant de types d'énumération différents est invalide.

```
enum X {
    A,
    B,
}
enum Y {
    A,
    B,
    C,
}
const xa: number = X.A; // Valide
const ya: Y = 0; // Valide
X.A === Y.A; // Invalide
```

Les instances d'une classe sont soumises à une vérification de compatibilité de leurs membres privés et protégés :

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}

class Y {
    private a: string;
    constructor(value: string) {
```

```
        this.a = value;
    }
}

let x: X = new Y('y'); // Invalide
```

La comparaison ne tient pas compte des différences dans la hiérarchie d'héritage, par exemple :

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
class Y extends X {
    public a: string;
    constructor(value: string) {
        super(value);
        this.a = value;
    }
}
class Z {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
let x: X = new X('x');
let y: Y = new Y('y');
let z: Z = new Z('z');
x === y; // Valide
x === z; // Valide même si z appartient à une autre hiérarchie
d'héritage
```

Les génériques sont comparés d'après leurs structures, en fonction du type obtenu après application du paramètre

générique ; seul le résultat final est comparé comme un type non générique.

```
interface X<T> {
    a: T;
}
let x: X<number> = { a: 1 };
let y: X<string> = { a: 'a' };
x === y; // Invalide, car l'argument de type est utilisé dans la structure finale

interface X<T> {}
const x: X<number> = 1;
const y: X<string> = 'a';
x === y; // Valide, car l'argument de type n'est pas utilisé dans la structure finale
```

Lorsque les arguments de type des génériques ne sont pas spécifiés, tous les arguments non spécifiés sont traités comme des types « any » :

```
type X = <T>(x: T) => T;
type Y = <K>(y: K) => K;
let x: X = x => x;
let y: Y = y => y;
x = y; // Valide
```

À retenir :

```
let a: number = 1;
let b: number = 2;
a = b; // Valide, tout est assignable à lui-même

let c: any;
c = 1; // Valide, tous les types sont assignables à any

let d: unknown;
d = 1; // Valide, tous les types sont assignables à unknown
```

```
let e: unknown;
let e1: unknown = e; // Valide, unknown n'est assignable qu'à
lui-même et à any
let e2: any = e; // Valide
let e3: number = e; // Invalide

let f: never;
f = 1; // Invalide, rien n'est assignable à never

let g: void;
let g1: any;
g = 1; // Invalide, void n'est assignable à aucun autre type
que any, et réciproquement
g = g1; // Valide
```

Veuillez noter que lorsque « strictNullChecks » est activé, « null » et « undefined » sont traités de la même manière que « void » ; dans le cas contraire, ils sont semblables à « never ».

Les types comme ensembles

Dans TypeScript, un type est un ensemble de valeurs possibles. Cet ensemble est également appelé le domaine du type. Chaque valeur d'un type peut être considérée comme un élément d'un ensemble. Un type établit les contraintes que chaque élément de l'ensemble doit respecter pour être considéré comme un membre de cet ensemble. La tâche principale de TypeScript consiste à contrôler et à vérifier si un ensemble est un sous-ensemble d'un autre.

TypeScript prend en charge différents types d'ensembles :

Terme ensembliste	TypeScript	Remarques

Ensemble vide	never	« never » ne contient rien en dehors de lui-même
Ensemble singleton	undefined / null / literal type	
Ensemble fini	boolean / union	
Ensemble infini	string / number / object	
Ensemble universel	any / unknown	Chaque élément appartient à « any » et chaque ensemble en est un sous-ensemble / « unknown » est l'équivalent de « any » qui préserve la sécurité du typage

Voici quelques exemples :

TypeScript	**Terme ensembliste**	**Exemple**
never	∅ (ensemble vide)	const x: never = 'x'; // Error: Type 'string' is not assignable to type 'never'
Literal type	Ensemble singleton	type X = 'X'; type Y = 7;
Valeur affectable à T	Valeur ∈ T (membre de)	type XY = 'X' | 'Y'; const x: XY = 'X';
T1 affectable à T2	T1 ⊆ T2 (sous-ensemble de)	type XY = 'X' | 'Y'; const x: XY = 'X'; const j: XY = 'J'; // Type '"J"' is not assignable to type 'XY'.
T1 extends T2	T1 ⊆ T2 (sous-ensemble de)	type X = 'X' extends string ? true : false;

T1 | T2	T1 ∪ T2 (union)	type XY = ‘X’ | ‘Y’; type JK = 1 | 2;
T1 & T2	T1 ∩ T2 (intersection)	type X = { a: string } type Y = { b: string } type XY = X & Y const x: XY = { a: ‘a’, b: ‘b’ }
unknown	Ensemble universel	const x: unknown = 1

Une union (T1 | T2) crée un ensemble plus large (les deux) :

```
type X = {
    a: string;
};
type Y = {
    b: string;
};
type XY = X | Y;
const r: XY = { a: 'a', b: 'x' }; // Valide
```

Une intersection (T1 & T2) crée un ensemble plus restreint (uniquement ce qui est partagé) :

```
type X = {
    a: string;
};
type Y = {
    a: string;
    b: string;
};
type XY = X & Y;
const r: XY = { a: 'a' }; // Invalide
const j: XY = { a: 'a', b: 'b' }; // Valide
```

Dans ce contexte, le mot-clé `extends` peut être considéré comme signifiant « sous-ensemble de ». Il établit une contrainte pour un type. Lorsque `extends` est utilisé avec un générique, il limite le paramètre de type générique à un type plus spécifique.

Veuillez noter qu'ici, `extends` n'a rien à voir avec l'héritage de classes au sens de la POO.

TypeScript utilise des types structurels et ne possède pas de hiérarchie nominale stricte. En réalité, comme dans l'exemple ci-dessous, deux types peuvent se chevaucher sans que l'un soit un sous-type de l'autre, car TypeScript prend en compte la structure, ou forme, des objets.

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
interface Z extends Y {
    c: string;
}
const z: Z = { a: 'a', b: 'b', c: 'c' };
interface X1 {
    a: string;
}
interface Y1 {
    a: string;
    b: string;
}
interface Z1 {
    a: string;
    b: string;
    c: string;
}
const z1: Z1 = { a: 'a', b: 'b', c: 'c' };
```

```
const r: Z1 = z; // Valide
```

Attribuer un type : déclarations et assertions de type

Dans TypeScript, un type peut être attribué de différentes manières :

Déclaration de type

Dans l'exemple suivant, nous utilisons x: X (« : Type ») pour déclarer un type pour la variable x.

```
type X = {
    a: string;
};

// Déclaration de type
const x: X = {
    a: 'a',
};
```

Si la variable ne respecte pas le format spécifié, TypeScript signale une erreur. Par exemple :

```
type X = {
    a: string;
};

const x: X = {
    a: 'a',
    b: 'b', // Erreur : un littéral objet ne peut spécifier que des propriétés connues
};
```

Assertion de type

Il est possible d'ajouter une assertion à l'aide du mot-clé `as`. Cela indique au compilateur que le développeur dispose de plus d'informations sur un type et supprime toutes les erreurs susceptibles de se produire.

Par exemple :

```
type X = {
    a: string;
};
const x = {
    a: 'a',
    b: 'b',
} as X;
```

Dans l'exemple ci-dessus, l'objet x fait l'objet d'une assertion de type X à l'aide du mot-clé as. Cela indique au compilateur TypeScript que l'objet est conforme au type spécifié, même s'il possède une propriété supplémentaire b absente de la définition du type.

Les assertions de type sont utiles lorsqu'un type plus spécifique doit être indiqué, notamment lors de l'utilisation du DOM. Par exemple :

```
const myInput = document.getElementById('my_input') as
HTMLInputElement;
```

Ici, l'assertion de type as HTMLInputElement sert à indiquer à TypeScript que le résultat de getElementById doit être traité comme un HTMLInputElement. Les assertions de type peuvent également être utilisées pour remapper des clés, comme le montre l'exemple ci-dessous avec les littéraux de gabarit :

```
type J<Type> = {
    [
        Property in keyof Type as `prefix_${string & Property}`
    ]: () => Type[Property];
};
type X = {
    a: string;
    b: number;
};
type Y = J<X>;
```

Dans cet exemple, le type `J<Type>` utilise un type mappé avec un littéral de gabarit pour remapper les clés de Type. Il crée de nouvelles propriétés en ajoutant « prefix_ » à chaque clé, et leurs valeurs correspondantes sont des fonctions qui renvoient les valeurs des propriétés d'origine.

Il convient de noter que lors de l'utilisation d'une assertion de type, TypeScript n'effectue pas de vérification des propriétés excédentaires. Il est donc généralement préférable d'utiliser une déclaration de type lorsque la structure de l'objet est connue à l'avance.

Déclarations ambiantes

Les déclarations ambiantes sont des fichiers qui décrivent les types du code JavaScript ; leur nom de fichier respecte le format `.d.ts.`. Elles sont généralement importées et utilisées pour annoter des bibliothèques JavaScript existantes ou pour ajouter des types aux fichiers JS existants de votre projet.

Les types de nombreuses bibliothèques courantes sont disponibles à l'adresse : https://github.com/DefinitelyTyped/DefinitelyTyped/

et peuvent être installés à l'aide de :

```
npm install --save-dev @types/library-name
```

Pour vos propres déclarations ambiantes, vous pouvez effectuer un import à l'aide de la référence « triple-slash » :

```
/// <reference path="./library-types.d.ts" />
```

Vous pouvez utiliser des déclarations ambiantes même dans des fichiers JavaScript à l'aide de `// @ts-check`.

Le mot-clé `declare` permet de définir des types pour du code JavaScript existant sans l'importer, en servant d'espace réservé aux types provenant d'un autre fichier ou disponibles globalement.

Vérification des propriétés et des propriétés excédentaires

TypeScript repose sur un système de types structurel, mais la vérification des propriétés excédentaires est une fonctionnalité de TypeScript qui lui permet de vérifier si un objet possède exactement les propriétés spécifiées dans le type.

La vérification des propriétés excédentaires est effectuée lors de l'affectation de littéraux d'objet à des variables ou lorsqu'ils sont passés comme arguments à un paramètre de fonction, par exemple.

```
type X = {
    a: string;
};
const y = { a: 'a', b: 'b' };
```

```
const x: X = y; // Valide grâce au typage structurel
const w: X = { a: 'a', b: 'b' }; // Invalide en raison du contrôle des propriétés excédentaires
```

Types faibles

Un type est considéré comme faible lorsqu'il ne contient qu'un ensemble de propriétés toutes facultatives :

```
type X = {
    a?: string;
    b?: string;
};
```

TypeScript considère comme une erreur l'affectation d'une valeur à un type faible lorsqu'il n'existe aucun chevauchement ; par exemple, le code suivant génère une erreur :

```
type Options = {
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;

fn({ c: 'c' }); // Invalide
```

Bien que cela ne soit pas recommandé, il est possible, si nécessaire, de contourner cette vérification à l'aide d'une assertion de type :

```
type Options = {
    a?: string;
    b?: string;
};
const fn = (options: Options) => undefined;
fn({ c: 'c' } as Options); // Valide
```

Ou en ajoutant unknown à la signature d'index du type faible :

```
type Options = {
    [prop: string]: unknown;
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;
fn({ c: 'c' }); // Valide
```

Vérification stricte des littéraux d'objet (fraîcheur)

La vérification stricte des littéraux d'objet, parfois appelée « fraîcheur », est une fonctionnalité de TypeScript qui permet de détecter les propriétés excédentaires ou mal orthographiées qui passeraient autrement inaperçues lors des vérifications structurelles normales des types.

Lors de la création d'un littéral d'objet, le compilateur TypeScript le considère comme « frais ». Si le littéral d'objet est affecté à une variable ou passé en tant que paramètre, TypeScript génère une erreur si le littéral d'objet spécifie des propriétés qui n'existent pas dans le type cible.

Cependant, cette « fraîcheur » disparaît lorsqu'un littéral d'objet est élargi ou qu'une assertion de type est utilisée.

Voici quelques exemples pour illustrer ce comportement :

```
type X = { a: string };
type Y = { a: string; b: string };

let x: X;
x = { a: 'a', b: 'b' }; // Contrôle de fraîcheur : affectation
invalide
```

```
var y: Y;
y = { a: 'a', bx: 'bx' }; // Contrôle de fraîcheur : affectation invalide

const fn = (x: X) => console.log(x.a);

fn(x);
fn(y); // Élargissement : aucune erreur, types structurellement compatibles

fn({ a: 'a', bx: 'b' }); // Contrôle de fraîcheur : argument invalide

let c: X = { a: 'a' };
let d: Y = { a: 'a', b: '' };
c = d; // Élargissement : aucun contrôle de fraîcheur
```

Inférence de type

TypeScript peut inférer les types lorsqu'aucune annotation n'est fournie lors de :

- L'initialisation d'une variable.
- L'initialisation d'un membre.
- La définition de valeurs par défaut pour les paramètres.
- La détermination du type de retour d'une fonction.

Par exemple :

```
let x = 'x'; // Le type inféré est string
```

Le compilateur TypeScript analyse la valeur ou l'expression et détermine son type à partir des informations disponibles.

Inférences plus avancées

Lorsque plusieurs expressions interviennent dans l'inférence de type, TypeScript recherche le « meilleur type commun ». Par exemple :

```
let x = [1, 'x', 1, null]; // Le type inféré est : (string | number | null)[]
```

Si le compilateur ne peut pas trouver le meilleur type commun, il renvoie un type union. Par exemple :

```
let x = [new RegExp('x'), new Date()]; // Le type inféré est : (RegExp | Date)[]
```

TypeScript utilise le « typage contextuel » en fonction de l'emplacement de la variable pour inférer les types. Dans l'exemple suivant, le compilateur sait que `e` est de type `MouseEvent` grâce au type de l'événement `click` défini dans le fichier lib.d.ts, qui contient des déclarations ambiantes pour diverses constructions JavaScript courantes et le DOM :

```
window.addEventListener('click', function (e) {}); // Le type inféré de e est MouseEvent
```

Élargissement des types

L'élargissement de type est le processus par lequel TypeScript attribue un type à une variable initialisée sans annotation de type. Il permet de passer de types étroits à des types plus larges, mais pas l'inverse. Dans l'exemple suivant :

```
let x = 'x'; // TypeScript infère string, un type large
let y: 'y' | 'x' = 'y'; // Le type de y est une union de types littéraux
y = x; // Invalide : le type 'string' n'est pas assignable au type '"x" | "y"'.
```

TypeScript attribue `string` à `x` à partir de l'unique valeur fournie lors de l'initialisation (`x`) ; il s'agit d'un exemple d'élargissement.

TypeScript permet de contrôler le processus d'élargissement, par exemple en utilisant « const ».

Const

L'utilisation du mot-clé `const` lors de la déclaration d'une variable entraîne une inférence de type plus étroite dans TypeScript.

Par exemple :

```
const x = 'x'; // TypeScript infère le type de x comme 'x', un type plus étroit
let y: 'y' | 'x' = 'y';
y = x; // Valide : le type de x est inféré comme 'x'
```

En déclarant la variable x avec `const`, son type est restreint à la valeur littérale spécifique 'x'. Comme le type de x est plus étroit, il peut être affecté à la variable y sans erreur. Le type peut être inféré ainsi, car les variables `const` ne peuvent pas être réaffectées. Leur type peut donc être restreint à un type littéral spécifique, ici le type littéral 'x'.

Modificateur Const sur les paramètres de type

Depuis la version 5.0 de TypeScript, il est possible de spécifier l'attribut `const` sur un paramètre de type générique. Cela permet d'inférer le type le plus précis possible. Voyons un exemple sans utiliser `const` :

```
function identity<T>(value: T) {
    // Pas de const ici
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // Le type inféré
est : { a: string; b: string; }
```

Comme vous pouvez le constater, les propriétés `a` et `b` sont inférées avec un type `string`.

Voyons maintenant la différence avec la version utilisant `const` :

```
function identity<const T>(value: T) {
     // Utilisation du modificateur const sur les paramètres de
type
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // Le type inféré
est : { a: "a"; b: "b"; }
```

Nous pouvons maintenant constater que les propriétés `a` et `b` sont inférées comme des littéraux de chaîne plutôt que comme de simples types `string`.

Assertion Const

Cette fonctionnalité permet de déclarer une variable avec un type littéral plus précis à partir de sa valeur d'initialisation, ce qui indique au compilateur que la valeur doit être traitée comme un littéral immuable. Voici quelques exemples :

Sur une seule propriété :

```
const v = {
    x: 3 as const,
};
v.x = 3;
```

Sur un objet entier :

```
const v = {
    x: 1,
    y: 2,
} as const;
```

Cela peut être particulièrement utile lors de la définition du type d'un tuple :

```
const x = [1, 2, 3]; // number[]
const y = [1, 2, 3] as const; // Tuple readonly [1, 2, 3]
```

Annotation de type explicite

Nous pouvons être explicites et fournir un type. Dans l'exemple suivant, la propriété `x` est de type `number` :

```
const v = {
    x: 1, // Type inféré : number (élargissement)
};
v.x = 3; // Valide
```

Nous pouvons rendre l'annotation de type plus précise en utilisant une union de types littéraux :

```
const v: { x: 1 | 2 | 3 } = {
    x: 1, // x est maintenant une union de types littéraux : 1
| 2 | 3
};
v.x = 3; // Valide
v.x = 100; // Invalide
```

Réduction

La réduction de type est le processus par lequel TypeScript réduit un type général à un type plus spécifique. Cela se produit

lorsque TypeScript analyse le code et détermine que certaines conditions ou opérations permettent d'affiner les informations de type.

La réduction des types peut se produire de différentes manières, notamment avec :

Conditions

En utilisant des instructions conditionnelles, telles que `if` ou `switch`, TypeScript peut réduire le type en fonction du résultat de la condition. Par exemple :

```
let x: number | undefined = 10;

if (x !== undefined) {
    x += 100; // Le type est number, après avoir été affiné par la condition
}
```

Lever une exception ou retourner une valeur

Lever une exception ou effectuer un retour anticipé depuis une branche peut aider TypeScript à réduire un type. Par exemple :

```
let x: number | undefined = 10;

if (x === undefined) {
    throw 'error';
}
x += 100;
```

Parmi les autres façons de réduire les types dans TypeScript figurent :

- L'opérateur `instanceof` : utilisé pour vérifier si un objet est une instance d'une classe spécifique.
- L'opérateur `in` : utilisé pour vérifier si une propriété existe dans un objet.
- L'opérateur `typeof` : utilisé pour vérifier le type d'une valeur lors de l'exécution.
- Les fonctions intégrées telles que `Array.isArray()` : utilisées pour vérifier si une valeur est un tableau.

Union discriminée

L'utilisation d'une « union discriminée » est un modèle dans TypeScript où une « étiquette » explicite est ajoutée aux objets afin de distinguer les différents types d'une union. Ce modèle est également appelé « union étiquetée ». Dans l'exemple suivant, l'« étiquette » est représentée par la propriété « type » :

```
type A = { type: 'type_a'; value: number };
type B = { type: 'type_b'; value: string };

const x = (input: A | B): string | number => {
    switch (input.type) {
        case 'type_a':
            return input.value + 100; // le type est A
        case 'type_b':
            return input.value + 'extra'; // le type est B
    }
};
```

Gardes de type définies par l'utilisateur

Lorsque TypeScript ne parvient pas à déterminer un type, il est possible d'écrire une fonction auxiliaire appelée « garde de type définie par l'utilisateur ». Dans l'exemple suivant, nous

utiliserons un prédicat de type pour réduire le type après l'application d'un filtrage :

```
const data = ['a', null, 'c', 'd', null, 'f'];

const r1 = data.filter(x => x != null); // Le type est (string | null)[], TypeScript n'a pas pu l'inférer correctement

const isValid = (item: string | null): item is string => item !== null; // Garde de type personnalisé

const r2 = data.filter(isValid); // Le type est maintenant correctement défini comme string[], grâce au garde de type à prédicat qui a permis de l'affiner
```

Réduction avec switch-true

TypeScript 5.3 ajoute la réduction avec switch-true, qui permet de remplacer des chaînes if/else peu lisibles par switch (true) utilisant des conditions booléennes. Elle améliore la lisibilité tout en continuant à réduire les types. Elle s'apparente à la correspondance de motifs, mais en plus simple.

```
function classify(x: unknown) {
    switch (true) {
        case typeof x === 'string':
            return `"${x.toUpperCase()}"`;
        case typeof x === 'number':
            return x > 0 ? 'positive' : 'negative';
        case Array.isArray(x):
            return `[${x.length} items]`;
        default:
            return 'something else';
    }
}
```

Types primitifs

TypeScript prend en charge 7 types primitifs. Un type de données primitif désigne un type qui n'est pas un objet et auquel aucune méthode n'est associée. Dans TypeScript, tous les types primitifs sont immuables, ce qui signifie que leurs valeurs ne peuvent plus être modifiées une fois affectées.

string

Le type primitif `string` stocke des données textuelles, et sa valeur est toujours entourée de guillemets simples ou doubles.

```
const x: string = 'x';
const y: string = 'y';
```

Les chaînes peuvent s'étendre sur plusieurs lignes lorsqu'elles sont entourées du caractère accent grave (`) :

```
let sentence: string = `xxx,
   yyy`;
```

boolean

Le type de données `boolean` dans TypeScript stocke une valeur binaire, soit `true`, soit `false`.

```
const isReady: boolean = true;
```

number

Dans TypeScript, un type de données `number` est représenté par une valeur à virgule flottante sur 64 bits. Un type `number` peut représenter des nombres entiers et des nombres fractionnaires.

TypeScript prend également en charge les notations hexadécimale, binaire et octale, par exemple :

```
const decimal: number = 10;
const hexadecimal: number = 0xa00d; // L'hexadécimal commence par 0x
const binary: number = 0b1010; // Le binaire commence par 0b
const octal: number = 0o633; // L'octal commence par 0o
```

bigint

Un `bigint` représente des valeurs entières qui peuvent être supérieures à l'entier maximal sûr pris en charge par `number`, à savoir 2^53 - 1.

Un `bigint` peut être créé en appelant la fonction intégrée `BigInt()` ou en ajoutant `n` à la fin de n'importe quel littéral numérique entier :

```
const x: bigint = BigInt(9007199254740991);
const y: bigint = 9007199254740991n;
```

Notes :

- Les valeurs `bigint` ne peuvent pas être mélangées avec `number` ni être utilisées avec l'objet intégré `Math` ; elles doivent être converties dans le même type.
- Les valeurs `bigint` ne sont disponibles que si la configuration de la cible est ES2020 ou une version ultérieure.

Symbol

Les symboles sont des identifiants uniques qui peuvent être utilisés comme clés de propriété dans les objets afin d'éviter les

conflits de noms.

```
type Obj = {
    [sym: symbol]: number;
};

const a = Symbol('a');
const b = Symbol('b');
let obj: Obj = {};
obj[a] = 123;
obj[b] = 456;

console.log(obj[a]); // 123
console.log(obj[b]); // 456
```

null et undefined

Les types `null` et `undefined` représentent tous deux l'absence de valeur.

Le type `undefined` signifie que la valeur n'est ni affectée ni initialisée, ou indique une absence involontaire de valeur.

Le type `null` signifie que nous savons que le champ n'a pas de valeur, que la valeur est donc indisponible, et indique une absence intentionnelle de valeur.

Array

Un `array` est un type de données pouvant stocker plusieurs valeurs, de même type ou non. Il peut être défini à l'aide de la syntaxe suivante :

```
const x: string[] = ['a', 'b'];
const y: Array<string> = ['a', 'b'];
const j: Array<string | number> = ['a', 1, 'b', 2]; // Union
```

TypeScript prend en charge les tableaux en lecture seule avec la syntaxe suivante :

```
const x: readonly string[] = ['a', 'b']; // Modificateur readonly
const y: ReadonlyArray<string> = ['a', 'b'];
const j: ReadonlyArray<string | number> = ['a', 1, 'b', 2];
j.push('x'); // Invalide
```

TypeScript prend en charge les tuples et les tuples en lecture seule :

```
const x: [string, number] = ['a', 1];
const y: readonly [string, number] = ['a', 1];
```

any

Le type de données any représente littéralement « n'importe quelle » valeur et constitue le type par défaut lorsque TypeScript ne peut pas inférer le type ou que celui-ci n'est pas spécifié.

Lors de l'utilisation de any, le compilateur TypeScript ignore la vérification des types. Il n'y a donc aucune sécurité des types lorsque any est utilisé. En règle générale, n'utilisez pas any pour faire taire le compilateur lorsqu'une erreur se produit ; concentrez-vous plutôt sur sa correction, car l'utilisation de any permet de rompre les contrats et de perdre les avantages de l'autocomplétion de TypeScript.

Le type any peut être utile lors d'une migration progressive de JavaScript vers TypeScript, car il permet de faire taire le compilateur.

Pour les nouveaux projets, utilisez l'option de configuration TypeScript `noImplicitAny`, qui permet à TypeScript de signaler les erreurs aux endroits où `any` est utilisé ou inféré.

Le type `any` est généralement une source d'erreurs susceptible de masquer de véritables problèmes dans vos types. Évitez autant que possible de l'utiliser.

Annotations de type

Pour les variables déclarées avec `var`, `let` et `const`, il est possible d'ajouter facultativement un type :

```
const x: number = 1;
```

TypeScript infère efficacement les types, en particulier les plus simples. Ces déclarations ne sont donc pas nécessaires dans la plupart des cas.

Dans les fonctions, il est possible d'ajouter des annotations de type aux paramètres :

```
function sum(a: number, b: number) {
    return a + b;
}
```

Voici un exemple utilisant une fonction anonyme (également appelée fonction lambda) :

```
const sum = (a: number, b: number) => a + b;
```

Ces annotations peuvent être omises lorsqu'une valeur par défaut est fournie pour un paramètre :

```
const sum = (a = 10, b: number) => a + b;
```

Des annotations de type de retour peuvent être ajoutées aux fonctions :

```
const sum = (a = 10, b: number): number => a + b;
```

C'est particulièrement utile pour les fonctions plus complexes, car écrire le type de retour avant l'implémentation peut vous aider à concevoir la fonction.

De manière générale, envisagez d'annoter les signatures de type, mais pas les variables locales au corps, et ajoutez toujours des types aux littéraux d'objet.

Propriétés facultatives

Un objet peut spécifier des propriétés facultatives en ajoutant un point d'interrogation `?` à la fin du nom de la propriété :

```
type X = {
    a: number;
    b?: number; // Facultatif
};
```

Il est possible de spécifier une valeur par défaut lorsqu'une propriété est facultative :

```
type X = {
    a: number;
    b?: number;
};
const x = ({ a, b = 100 }: X) => a + b;
```

Propriétés en lecture seule

Il est possible d'empêcher l'écriture dans une propriété en utilisant le modificateur `readonly`, qui garantit que la propriété ne peut pas être réécrite, mais n'offre aucune garantie d'immuabilité totale :

```
interface Y {
    readonly a: number;
}

type X = {
    readonly a: number;
};

type J = Readonly<{
    a: number;
}>;

type K = {
    readonly [index: number]: string;
};
```

Signatures d'index

Dans TypeScript, nous pouvons utiliser `string`, `number` et `symbol` comme signatures d'index :

```
type K = {
    [name: string | number]: string;
};
const k: K = { x: 'x', 1: 'b' };
console.log(k['x']);
console.log(k[1]);
console.log(k['1']); // Même résultat que k[1]
```

Veuillez noter que JavaScript convertit automatiquement un index de type `number` en index de type `string`. Ainsi, `k[1]` et `k["1"]`

renvoient la même valeur.

Extension des types

Il est possible d'étendre une `interface` (en copiant les membres d'un autre type) :

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
```

Il est également possible d'étendre plusieurs types :

```
interface A {
    a: string;
}
interface B {
    b: string;
}
interface Y extends A, B {
    y: string;
}
```

Le mot-clé `extends` fonctionne uniquement avec les interfaces et les classes ; pour les types, utilisez une intersection :

```
type A = {
    a: number;
};
type B = {
    b: number;
};
type C = A & B;
```

Il est possible d'étendre un type à l'aide d'une interface, mais pas l'inverse :

```
type A = {
    a: string;
};
interface B extends A {
    b: string;
}
```

Types littéraux

Un type littéral est un ensemble à un seul élément au sein d'un type collectif ; il définit une valeur très précise qui est une primitive JavaScript.

Les types littéraux dans TypeScript sont les nombres, les chaînes et les booléens.

Exemples de littéraux :

```
const a = 'a'; // Type littéral de chaîne
const b = 1; // Type littéral numérique
const c = true; // Type littéral booléen
```

Les types littéraux de chaîne, numériques et booléens sont utilisés dans les unions, les gardes de type et les alias de type. Dans l'exemple suivant, vous pouvez voir un alias de type union. `O` se compose uniquement des valeurs spécifiées ; aucune autre chaîne n'est valide :

```
type O = 'a' | 'b' | 'c';
```

Inférence des littéraux

L'inférence des littéraux est une fonctionnalité de TypeScript qui permet d'inférer le type d'une variable ou d'un paramètre à partir de sa valeur.

Dans l'exemple suivant, nous pouvons voir que TypeScript considère `x` comme un type littéral, car sa valeur ne peut plus être modifiée ultérieurement, tandis que `y` est inféré comme une chaîne puisqu'il peut être modifié ultérieurement.

```
const x = 'x'; // Type littéral de 'x', car cette valeur ne peut pas être modifiée
let y = 'y'; // Type string, car cette valeur peut être modifiée
```

Dans l'exemple suivant, nous pouvons voir que `o.x` a été inféré comme un `string` (et non comme le littéral `a`), car TypeScript considère que la valeur peut être modifiée ultérieurement.

```
type X = 'a' | 'b';

let o = {
    x: 'a', // Il s'agit d'une chaîne de type plus large
};

const fn = (x: X) => `${x}-foo`;

console.log(fn(o.x)); // Un argument de type 'string' n'est pas assignable à un paramètre de type 'X'
```

Comme vous pouvez le constater, le code génère une erreur lors du passage de `o.x` à `fn`, car X est un type plus étroit.

Nous pouvons résoudre ce problème en utilisant une assertion de type avec `const` ou le type `X` :

```ts
let o = {
    x: 'a' as const,
};
```

ou :

```ts
let o = {
    x: 'a' as X,
};
```

strictNullChecks

`strictNullChecks` est une option du compilateur TypeScript qui impose une vérification stricte des valeurs nulles. Lorsque cette option est activée, `null` ou `undefined` ne peuvent être affectés aux variables et aux paramètres que si ceux-ci ont été explicitement déclarés avec ce type au moyen du type union `null | undefined`. Si une variable ou un paramètre n'est pas explicitement déclaré comme nullable, TypeScript génère une erreur afin d'éviter de potentielles erreurs lors de l'exécution.

Énumérations

Dans TypeScript, un `enum` est un ensemble de valeurs constantes nommées.

```ts
enum Color {
    Red = '#ff0000',
    Green = '#00ff00',
    Blue = '#0000ff',
}
```

Les énumérations peuvent être définies de différentes manières :

Énumérations numériques

Dans TypeScript, une énumération numérique est une énumération dans laquelle chaque constante reçoit une valeur numérique, en commençant par 0 par défaut.

```
enum Size {
    Small, // la valeur commence à 0
    Medium,
    Large,
}
```

Il est possible de spécifier des valeurs personnalisées en les affectant explicitement :

```
enum Size {
    Small = 10,
    Medium,
    Large,
}
console.log(Size.Medium); // 11
```

Énumérations de chaînes

Dans TypeScript, une énumération de chaînes est une énumération dans laquelle chaque constante reçoit une valeur de type chaîne.

```
enum Language {
    English = 'EN',
    Spanish = 'ES',
}
```

Remarque : TypeScript autorise l'utilisation d'énumérations hétérogènes dans lesquelles des membres de type chaîne et de type numérique peuvent coexister.

Énumérations constantes

Dans TypeScript, une énumération constante est un type particulier d'énumération dont toutes les valeurs sont connues au moment de la compilation et incorporées partout où l'énumération est utilisée, ce qui produit un code plus efficace.

```
const enum Language {
    English = 'EN',
    Spanish = 'ES',
}
console.log(Language.English);
```

Sera compilé en :

```
console.log('EN' /* Language.English */);
```

Notes : Les énumérations constantes ont des valeurs codées en dur, ce qui efface l'énumération. Cela peut être plus efficace dans les bibliothèques autonomes, mais n'est généralement pas souhaitable. En outre, les énumérations constantes ne peuvent pas avoir de membres calculés.

Mappage inverse

Dans TypeScript, les mappages inverses dans les énumérations désignent la possibilité de récupérer le nom d'un membre de l'énumération à partir de sa valeur. Par défaut, les membres d'une énumération disposent de mappages directs du nom vers la valeur, mais des mappages inverses peuvent être créés en définissant explicitement les valeurs de chaque membre. Les mappages inverses sont utiles lorsque vous devez rechercher un membre d'une énumération par sa valeur ou parcourir tous ses membres. Notez que seuls les membres numériques d'une

énumération génèrent des mappages inverses ; les membres de type chaîne n'en génèrent aucun.

L'énumération suivante :

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
```

Est compilée en :

```
'use strict';
var Grade;
(function (Grade) {
    Grade[(Grade['A'] = 90)] = 'A';
    Grade[(Grade['B'] = 80)] = 'B';
    Grade[(Grade['C'] = 70)] = 'C';
    Grade['F'] = 'fail';
})(Grade || (Grade = {}));
```

Par conséquent, le mappage des valeurs vers les clés fonctionne pour les membres numériques d'une énumération, mais pas pour les membres de type chaîne :

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
const myGrade = Grade.A;
console.log(Grade[myGrade]); // A
console.log(Grade[90]); // A

const failGrade = Grade.F;
```

```
console.log(failGrade); // fail
console.log(Grade[failGrade]); // L'élément a implicitement le type 'any', car l'expression d'index n'est pas de type 'number'.
```

Énumérations ambiantes

Dans TypeScript, une énumération ambiante est un type d'énumération défini dans un fichier de déclaration (*.d.ts) sans implémentation associée. Elle permet de définir un ensemble de constantes nommées pouvant être utilisées de manière sûre du point de vue des types dans différents fichiers, sans avoir à importer les détails d'implémentation dans chaque fichier.

Membres calculés et constants

Dans TypeScript, un membre calculé est un membre d'une énumération dont la valeur est calculée lors de l'exécution, tandis qu'un membre constant est un membre dont la valeur est définie au moment de la compilation et ne peut pas être modifiée lors de l'exécution. Les membres calculés sont autorisés dans les énumérations classiques, tandis que les membres constants sont autorisés à la fois dans les énumérations classiques et constantes.

```
// Membres constants
enum Color {
    Red = 1,
    Green = 5,
    Blue = Red + Green,
}
console.log(Color.Blue); // 6 généré à la compilation

// Membres calculés
enum Color {
```

```
    Red = 1,
    Green = Math.pow(2, 2),
    Blue = Math.floor(Math.random() * 3) + 1,
}
console.log(Color.Blue);   // nombre aléatoire généré à
l'exécution
```

Les énumérations sont représentées par des unions composées des types de leurs membres. La valeur de chaque membre peut être déterminée au moyen d'expressions constantes ou non constantes, les membres possédant des valeurs constantes recevant des types littéraux. Pour illustrer cela, considérons la déclaration du type E et de ses sous-types E.A, E.B et E.C. Dans ce cas, E représente l'union E.A | E.B | E.C.

```
const identity = (value: number) => value;

enum E {
    A = 2 * 5, // Littéral numérique
    B = 'bar', // Littéral de chaîne
    C = identity(42), // Valeur calculée opaque
}

console.log(E.C); //42
```

Réduction de type

La réduction de type dans TypeScript est le processus qui consiste à affiner le type d'une variable au sein d'un bloc conditionnel. Elle est utile lorsque vous travaillez avec des types union, où une variable peut avoir plusieurs types.

TypeScript reconnaît plusieurs façons de réduire le type :

Gardes de type typeof

La garde de type typeof est une garde de type spécifique à TypeScript qui vérifie le type d'une variable à partir de son type JavaScript intégré.

```
const fn = (x: number | string) => {
    if (typeof x === 'number') {
        return x + 1; // x est de type number
    }
    return -1;
};
```

Réduction par véracité

Dans TypeScript, la réduction par véracité consiste à vérifier si une variable est évaluée comme vraie ou fausse afin de réduire son type en conséquence.

```
const toUpperCase = (name: string | null) => {
    if (name) {
        return name.toUpperCase();
    } else {
        return null;
    }
};
```

Réduction par égalité

Dans TypeScript, la réduction par égalité consiste à vérifier si une variable est égale ou non à une valeur spécifique afin de réduire son type en conséquence.

Elle est utilisée conjointement avec les instructions `switch` et les opérateurs d'égalité tels que `===`, `!==`, `==` et `!=` pour réduire les types.

```
const checkStatus = (status: 'success' | 'error') => {
    switch (status) {
        case 'success':
            return true;
        case 'error':
            return null;
    }
};
```

Réduction avec l'opérateur in

Dans TypeScript, la réduction avec l'opérateur `in` permet de réduire le type d'une variable selon qu'une propriété existe ou non dans le type de la variable.

```
type Dog = {
    name: string;
    breed: string;
};

type Cat = {
    name: string;
    likesCream: boolean;
};

const getAnimalType = (pet: Dog | Cat) => {
    if ('breed' in pet) {
        return 'dog';
    } else {
        return 'cat';
    }
};
```

Réduction avec instanceof

Dans TypeScript, la réduction avec l'opérateur `instanceof` permet de réduire le type d'une variable à partir de sa fonction

constructeur, en vérifiant si un objet est une instance d'une certaine classe ou interface.

```
class Square {
    constructor(public width: number) {}
}
class Rectangle {
    constructor(
        public width: number,
        public height: number
    ) {}
}
function area(shape: Square | Rectangle) {
    if (shape instanceof Square) {
        return shape.width * shape.width;
    } else {
        return shape.width * shape.height;
    }
}
const square = new Square(5);
const rectangle = new Rectangle(5, 10);
console.log(area(square)); // 25
console.log(area(rectangle)); // 50
```

Affectations

Dans TypeScript, la réduction de type à l'aide d'affectations permet de réduire le type d'une variable à partir de la valeur qui lui est affectée. Lorsqu'une valeur est affectée à une variable, TypeScript infère son type à partir de cette valeur et réduit le type de la variable pour qu'il corresponde au type inféré.

```
let value: string | number;
value = 'hello';
if (typeof value === 'string') {
```

```
    console.log(value.toUpperCase());
}
value = 42;
if (typeof value === 'number') {
    console.log(value.toFixed(2));
}
```

Analyse du flux de contrôle

Dans TypeScript, l'analyse du flux de contrôle permet d'analyser statiquement le déroulement du code afin d'inférer les types des variables. Le compilateur peut ainsi réduire les types de ces variables selon les besoins, à partir des résultats de l'analyse.

Avant TypeScript 4.4, l'analyse du flux de code ne s'appliquait qu'au code situé dans une instruction if. Depuis TypeScript 4.4, elle peut également s'appliquer aux expressions conditionnelles et aux accès à des propriétés discriminantes référencés indirectement par des variables const.

Par exemple :

```
const f1 = (x: unknown) => {
    const isString = typeof x === 'string';
    if (isString) {
        x.length;
    }
};

const f2 = (
     obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar:
number }
) => {
    const isFoo = obj.kind === 'foo';
    if (isFoo) {
```

```
        obj.foo;
    } else {
        obj.bar;
    }
};
```

Quelques exemples dans lesquels la réduction ne se produit pas :

```
const f1 = (x: unknown) => {
    let isString = typeof x === 'string';
    if (isString) {
            x.length; // Erreur, aucun affinement car isString
n'est pas const
    }
};

const f6 = (
     obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar:
number }
) => {
    const isFoo = obj.kind === 'foo';
    obj = obj;
    if (isFoo) {
            obj.foo; // Erreur, aucun affinement car obj est
réaffecté dans le corps de la fonction
    }
};
```

Remarque : jusqu'à cinq niveaux d'indirection sont analysés dans les expressions conditionnelles.

Prédicats de type

Dans TypeScript, les prédicats de type sont des fonctions qui renvoient une valeur booléenne et servent à réduire le type d'une variable à un type plus spécifique.

```
const isString = (value: unknown): value is string => typeof
value === 'string';

const foo = (bar: unknown) => {
    if (isString(bar)) {
        console.log(bar.toUpperCase());
    } else {
        console.log('not a string');
    }
};
```

TypeScript 5.5 infère automatiquement les prédicats de type (comme `x is T`) dans des fonctions telles que `.filter`. Il sait ainsi quand des valeurs comme undefined sont supprimées, ce qui permet d'obtenir des types plus précis et moins d'erreurs. Cela fonctionne pour les vérifications explicites (par exemple, `x !== undefined`), mais pas pour les vérifications ambiguës comme `!!x`.

```
const nums = [1, null, 2].filter(x => x !== null);
```

Unions discriminées

En TypeScript, les unions discriminées sont un type d'union qui utilise une propriété commune, appelée discriminant, pour restreindre l'ensemble des types possibles de l'union.

```
type Square = {
    kind: 'square'; // Discriminant
    size: number;
};

type Circle = {
    kind: 'circle'; // Discriminant
    radius: number;
};
```

```ts
type Shape = Square | Circle;

const area = (shape: Shape) => {
    switch (shape.kind) {
        case 'square':
            return Math.pow(shape.size, 2);
        case 'circle':
            return Math.PI * Math.pow(shape.radius, 2);
    }
};

const square: Square = { kind: 'square', size: 5 };
const circle: Circle = { kind: 'circle', radius: 2 };

console.log(area(square)); // 25
console.log(area(circle)); // 12.566370614359172
```

Le type never

Lorsqu'une variable est restreinte à un type qui ne peut contenir aucune valeur, le compilateur TypeScript en déduit que la variable doit être de type `never`. En effet, le type never représente une valeur qui ne peut jamais être produite.

```ts
const printValue = (val: string | number) => {
    if (typeof val === 'string') {
        console.log(val.toUpperCase());
    } else if (typeof val === 'number') {
        console.log(val.toFixed(2));
    } else {
        // val a ici le type never, car il ne peut jamais être autre chose qu'une chaîne ou un nombre
        const neverVal: never = val;
        console.log(`Unexpected value: ${neverVal}`);
    }
};
```

Vérification de l'exhaustivité

La vérification de l'exhaustivité est une fonctionnalité de TypeScript qui garantit que tous les cas possibles d'une union discriminée sont traités dans une instruction `switch` ou une instruction `if`.

```
type Direction = 'up' | 'down';

const move = (direction: Direction) => {
    switch (direction) {
        case 'up':
            console.log('Moving up');
            break;
        case 'down':
            console.log('Moving down');
            break;
        default:
            const exhaustiveCheck: never = direction;
                console.log(exhaustiveCheck); // Cette ligne ne
sera jamais exécutée
    }
};
```

Le type `never` permet de garantir que le cas par défaut est exhaustif et que TypeScript signalera une erreur si une nouvelle valeur est ajoutée au type Direction sans être traitée dans l'instruction switch.

Types d'objets

En TypeScript, les types d'objet décrivent la forme d'un objet. Ils précisent les noms et les types des propriétés de l'objet, ainsi que le caractère obligatoire ou facultatif de ces propriétés.

En TypeScript, vous pouvez définir les types d'objet de deux manières principales :

Une interface définit la forme d'un objet en précisant les noms, les types et le caractère facultatif de ses propriétés.

```
interface User {
    name: string;
    age: number;
    email?: string;
}
```

Un alias de type, tout comme une interface, définit la forme d'un objet. Cependant, il peut également créer un nouveau type personnalisé fondé sur un type existant ou sur une combinaison de types existants. Cela inclut la définition de types union, de types intersection et d'autres types complexes.

```
type Point = {
    x: number;
    y: number;
};
```

Il est également possible de définir un type de manière anonyme :

```
const sum = (x: { a: number; b: number }) => x.a + x.b;
console.log(sum({ a: 5, b: 1 }));
```

Type tuple (anonyme)

Un type tuple est un type qui représente un tableau comportant un nombre fixe d'éléments et leurs types correspondants. Un type tuple impose un nombre précis d'éléments ainsi que leurs types respectifs, dans un ordre fixe. Les types tuple sont utiles

lorsque vous souhaitez représenter une collection de valeurs de types précis, où la position de chaque élément dans le tableau possède une signification particulière.

```
type Point = [number, number];
```

Type tuple nommé (étiqueté)

Les types tuple peuvent inclure des étiquettes ou des noms facultatifs pour chaque élément. Ces étiquettes améliorent la lisibilité et facilitent l'utilisation des outils, mais n'ont aucune incidence sur les opérations que vous pouvez effectuer avec ces éléments.

```
type T = string;
type Tuple1 = [T, T];
type Tuple2 = [a: T, b: T];
type Tuple3 = [a: T, T]; // Tuple nommé et tuple anonyme
```

Tuple de longueur fixe

Un tuple de longueur fixe est un type particulier de tuple qui impose un nombre fixe d'éléments de types précis et interdit toute modification de la longueur du tuple une fois celui-ci défini.

Les tuples de longueur fixe sont utiles lorsque vous devez représenter une collection de valeurs comportant un nombre précis d'éléments de types précis, et que vous souhaitez garantir que la longueur et les types du tuple ne puissent pas être modifiés par inadvertance.

```
const x = [10, 'hello'] as const;
x.push(2); // Erreur
```

Type union

Un type union représente une valeur qui peut appartenir à l'un de plusieurs types. Les types union sont indiqués à l'aide du symbole `|` entre chaque type possible.

```
let x: string | number;
x = 'hello'; // Valide
x = 123; // Valide
```

Types intersection

Un type intersection représente une valeur qui possède toutes les propriétés de deux types ou plus. Les types intersection sont indiqués à l'aide du symbole `&` entre chaque type.

```
type X = {
    a: string;
};

type Y = {
    b: string;
};

type J = X & Y; // Intersection

const j: J = {
    a: 'a',
    b: 'b',
};
```

Indexation de type

L'indexation de type désigne la possibilité de définir des types pouvant être indexés par une clé qui n'est pas connue à

l'avance, en utilisant une signature d'index pour préciser le type des propriétés qui ne sont pas déclarées explicitement.

```
type Dictionary<T> = {
    [key: string]: T;
};
const myDict: Dictionary<string> = { a: 'a', b: 'b' };
console.log(myDict['a']); // Renvoie a
```

Type à partir d'une valeur

En TypeScript, un type à partir d'une valeur désigne l'inférence automatique d'un type à partir d'une valeur ou d'une expression.

```
const x = 'x'; // TypeScript infère 'x' comme un littéral de chaîne avec 'const' (immuable), mais l'élargit à 'string' avec 'let' (réaffectable).
```

Type à partir de la valeur de retour d'une fonction

Le type à partir de la valeur de retour d'une fonction désigne la possibilité d'inférer automatiquement le type de retour d'une fonction en fonction de son implémentation. TypeScript peut ainsi déterminer le type de la valeur renvoyée par la fonction sans annotation de type explicite.

```
const add = (x: number, y: number) => x + y; // TypeScript peut inférer que le type de retour de la fonction est number
```

Type à partir d'un module

Le type à partir d'un module désigne la possibilité d'utiliser les valeurs exportées d'un module afin d'en inférer automatiquement les types. Lorsqu'un module exporte une valeur avec un type précis, TypeScript peut utiliser cette information pour inférer automatiquement le type de cette valeur lorsqu'elle est importée dans un autre module.

```
// calc.ts
export const add = (x: number, y: number) => x + y;
// index.ts
import { add } from 'calc';
const r = add(1, 2); // r est de type number
```

Types mappés

En TypeScript, les types mappés permettent de créer de nouveaux types à partir d'un type existant en transformant chaque propriété à l'aide d'une fonction de mappage. En mappant des types existants, vous pouvez créer de nouveaux types qui représentent les mêmes informations dans un format différent. Pour créer un type mappé, vous accédez aux propriétés d'un type existant à l'aide de l'opérateur `keyof`, puis vous les modifiez afin de produire un nouveau type. Dans l'exemple suivant :

```
type MyMappedType<T> = {
    [P in keyof T]: T[P][];
};
type MyType = {
    foo: string;
    bar: number;
};
type MyNewType = MyMappedType<MyType>;
const x: MyNewType = {
    foo: ['hello', 'world'],
```

```
    bar: [1, 2, 3],
};
```

nous définissons MyMappedType afin de parcourir les propriétés de T et de créer un nouveau type dont chaque propriété est un tableau du type d'origine. À partir de celui-ci, nous créons MyNewType pour représenter les mêmes informations que MyType, mais avec chaque propriété sous forme de tableau.

Modificateurs de types mappés

En TypeScript, les modificateurs de types mappés permettent de transformer les propriétés d'un type existant :

- `readonly` ou `+readonly` : rend une propriété du type mappé accessible en lecture seule.
- `-readonly` : rend une propriété du type mappé modifiable.
- `?` : désigne une propriété du type mappé comme facultative.

Exemples :

```
type ReadOnly<T> = { readonly [P in keyof T]: T[P] }; // Toutes les propriétés sont en lecture seule

type Mutable<T> = { -readonly [P in keyof T]: T[P] }; // Toutes les propriétés sont modifiables

type MyPartial<T> = { [P in keyof T]?: T[P] }; // Toutes les propriétés sont facultatives
```

Types conditionnels

Les types conditionnels permettent de créer un type qui dépend d'une condition, le type à créer étant déterminé par le résultat

de cette condition. Ils sont définis à l'aide du mot-clé `extends` et d'un opérateur ternaire afin de choisir conditionnellement entre deux types.

```
type IsArray<T> = T extends any[] ? true : false;

const myArray = [1, 2, 3];
const myNumber = 42;

type IsMyArrayAnArray = IsArray<typeof myArray>; // Type true
type IsMyNumberAnArray = IsArray<typeof myNumber>; // Type
false
```

Types conditionnels distributifs

Les types conditionnels distributifs permettent de distribuer un type sur une union de types en appliquant une transformation à chaque membre de l'union individuellement. Cela peut être particulièrement utile lorsque vous travaillez avec des types mappés ou des types d'ordre supérieur.

```
type Nullable<T> = T extends any ? T | null : never;
type NumberOrBool = number | boolean;
type NullableNumberOrBool = Nullable<NumberOrBool>; // number |
boolean | null
```

Inférence de type avec infer dans les types conditionnels

Le mot-clé `infer` est utilisé dans les types conditionnels afin d'inférer (extraire) le type d'un paramètre générique depuis un type qui en dépend. Il permet ainsi d'écrire des définitions de types plus souples et réutilisables.

```ts
type ElementType<T> = T extends (infer U)[] ? U : never;
type Numbers = ElementType<number[]>; // number
type Strings = ElementType<string[]>; // string
```

Types conditionnels prédéfinis

En TypeScript, les types conditionnels prédéfinis sont des types conditionnels intégrés fournis par le langage. Ils sont conçus pour effectuer des transformations de types courantes en fonction des caractéristiques d'un type donné.

`Exclude<UnionType, ExcludedType>` : ce type supprime de Type tous les types qui sont assignables à ExcludedType.

`Extract<Type, Union>` : ce type extrait de Union tous les types qui sont assignables à Type.

`NonNullable<Type>` : ce type supprime null et undefined de Type.

`ReturnType<Type>` : ce type extrait le type de retour d'une fonction Type.

`Parameters<Type>` : ce type extrait les types des paramètres d'une fonction Type.

`Required<Type>` : ce type rend obligatoires toutes les propriétés de Type.

`Partial<Type>` : ce type rend facultatives toutes les propriétés de Type.

`Readonly<Type>` : ce type rend accessibles en lecture seule toutes les propriétés de Type.

Types union de littéraux de gabarit

Les types union de littéraux de gabarit peuvent servir à fusionner et à manipuler du texte au sein du système de types, par exemple :

```
type Status = 'active' | 'inactive';
type Products = 'p1' | 'p2';
type ProductId = `id-${Products}-${Status}`; // "id-p1-active"
| "id-p1-inactive" | "id-p2-active" | "id-p2-inactive"
```

Type any

Le type `any` est un type spécial (supertype universel) qui peut représenter tout type de valeur (primitives, objets, tableaux, fonctions, erreurs, symboles). Il est souvent utilisé lorsque le type d'une valeur n'est pas connu au moment de la compilation, ou lorsque vous travaillez avec des valeurs provenant d'API ou de bibliothèques externes dépourvues de typages TypeScript.

En utilisant le type `any`, vous indiquez au compilateur TypeScript que les valeurs doivent être représentées sans aucune restriction. Pour améliorer la sécurité du typage dans votre code, tenez compte des recommandations suivantes :

- Limitez l'utilisation de `any` aux cas précis où le type est réellement inconnu.
- Ne renvoyez pas de types `any` depuis une fonction, car cela réduit la sécurité du typage dans le code qui l'utilise.
- À la place de `any`, utilisez `@ts-ignore` si vous devez faire taire le compilateur.

```
let value: any;
value = true; // Valide
```

```
value = 7; // Valide
```

Type unknown

En TypeScript, le type `unknown` représente une valeur dont le type est inconnu. Contrairement au type `any`, qui autorise tout type de valeur, `unknown` exige une vérification ou une assertion de type avant de pouvoir être utilisé d'une manière précise. Ainsi, aucune opération n'est autorisée sur un type `unknown` sans assertion ou restriction préalable vers un type plus précis.

Le type `unknown` est uniquement assignable à `any` et à `unknown` lui-même, et constitue une alternative à `any` qui préserve la sécurité du typage.

```
let value: unknown;

let value1: unknown = value; // Valide
let value2: any = value; // Valide
let value3: boolean = value; // Invalide
let value4: number = value; // Invalide

const add = (a: unknown, b: unknown): number | undefined =>
     typeof a === 'number' && typeof b === 'number' ? a + b :
undefined;
console.log(add(1, 2)); // 3
console.log(add('x', 2)); // undefined
```

Type void

Le type `void` sert à indiquer qu'une fonction ne renvoie aucune valeur.

```
const sayHello = (): void => {
    console.log('Hello!');
```

```
};
```

Type never

Le type `never` représente les valeurs qui ne se produisent jamais. Il sert à désigner des fonctions ou des expressions qui ne renvoient jamais de valeur ou lèvent une erreur.

Par exemple, une boucle infinie :

```
const infiniteLoop = (): never => {
    while (true) {
        // faire quelque chose
    }
};
```

Levée d'une erreur :

```
const throwError = (message: string): never => {
    throw new Error(message);
};
```

Le type `never` permet de garantir la sûreté des types et de détecter des erreurs potentielles dans votre code. Il aide TypeScript à analyser et à inférer des types plus précis lorsqu'il est utilisé avec d'autres types et des instructions de flux de contrôle, par exemple :

```
type Direction = 'up' | 'down';
const move = (direction: Direction): void => {
    switch (direction) {
        case 'up':
            // monter
            break;
        case 'down':
            // descendre
```

```
            break;
        default:
            const exhaustiveCheck: never = direction;
                        throw new Error(`Unhandled direction:
${exhaustiveCheck}`);
    }
};
```

Interface et type

Syntaxe courante

En TypeScript, les interfaces définissent la structure des objets en précisant les noms et les types des propriétés ou des méthodes qu'un objet doit posséder. La syntaxe courante pour définir une interface en TypeScript est la suivante :

```
interface InterfaceName {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
}
```

De même, pour une définition de type :

```
type TypeName = {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
};
```

`interface InterfaceName` ou `type TypeName` : définit le nom de l'interface. `property1 : Type1` : indique les propriétés de l'interface ainsi que leurs types correspondants. Plusieurs

propriétés peuvent être définies, chacune étant séparée par un point-virgule. `method1(arg1: ArgType1, arg2: ArgType2): ReturnType;` : indique les méthodes de l'interface. Les méthodes sont définies avec leur nom, suivi d'une liste de paramètres entre parenthèses et du type de retour. Plusieurs méthodes peuvent être définies, chacune étant séparée par un point-virgule.

Exemple d'interface :

```
interface Person {
    name: string;
    age: number;
    greet(): void;
}
```

Exemple de type :

```
type TypeName = {
    property1: string;
    method1(arg1: string, arg2: string): string;
};
```

En TypeScript, les types servent à définir la forme des données et à imposer la vérification des types. Il existe plusieurs syntaxes courantes pour définir les types en TypeScript, selon le cas d'utilisation précis. En voici quelques exemples :

Types de base

```
let myNumber: number = 123; // type number
let myBoolean: boolean = true; // type boolean
let myArray: string[] = ['a', 'b']; // tableau de chaînes
let myTuple: [string, number] = ['a', 123]; // tuple
```

Objets et interfaces

```
const x: { name: string; age: number } = { name: 'Simon', age:
7 };
```

Types union et intersection

```
type MyType = string | number; // Type union
let myUnion: MyType = 'hello'; // Peut être une chaîne
myUnion = 123; // Ou un nombre

type TypeA = { name: string };
type TypeB = { age: number };
type CombinedType = TypeA & TypeB; // Type intersection
let myCombined: CombinedType = { name: 'John', age: 25 }; //
Objet possédant les propriétés name et age
```

Types primitifs intégrés

TypeScript possède plusieurs types primitifs intégrés qui peuvent servir à définir des variables, des paramètres de fonction et des types de retour :

- `number` : représente les valeurs numériques, y compris les nombres entiers et à virgule flottante.
- `string` : représente les données textuelles
- `boolean` : représente les valeurs logiques, qui peuvent être vraies ou fausses.
- `null` : représente l'absence de valeur.
- `undefined` : représente une valeur qui n'a pas été affectée ou qui n'a pas été définie.
- `symbol` : représente un identifiant unique. Les symboles sont généralement utilisés comme clés pour les propriétés d'un objet.
- `bigint` : représente des nombres entiers de précision arbitraire.

- `any` : représente un type dynamique ou inconnu. Les variables de type any peuvent contenir des valeurs de n'importe quel type et ne sont pas soumises à la vérification des types.
- `void` : représente l'absence de tout type. Il est couramment utilisé comme type de retour des fonctions qui ne renvoient aucune valeur.
- `never` : représente un type pour les valeurs qui ne se produisent jamais. Il est généralement utilisé comme type de retour des fonctions qui lèvent une erreur ou entrent dans une boucle infinie.

Objets JS intégrés courants

TypeScript est un sur-ensemble de JavaScript ; il inclut tous les objets JavaScript intégrés couramment utilisés. Vous trouverez une liste exhaustive de ces objets sur le site de documentation du Mozilla Developer Network (MDN) : https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects

Voici une liste de quelques objets JavaScript intégrés couramment utilisés :

- Function
- Object
- Boolean
- Error
- Number
- BigInt
- Math
- Date

- String
- RegExp
- Array
- Map
- Set
- Promise
- Intl

Surcharges

En TypeScript, les surcharges de fonction permettent de définir plusieurs signatures de fonction pour un seul nom de fonction, ce qui permet de définir des fonctions pouvant être appelées de plusieurs manières. En voici un exemple :

```
// Surcharges
function sayHi(name: string): string;
function sayHi(names: string[]): string[];

// Implémentation
function sayHi(name: unknown): unknown {
    if (typeof name === 'string') {
        return `Hi, ${name}!`;
    } else if (Array.isArray(name)) {
        return name.map(name => `Hi, ${name}!`);
    }
    throw new Error('Invalid value');
}

sayHi('xx'); // Valide
sayHi(['aa', 'bb']); // Valide
```

Voici un autre exemple d'utilisation de surcharges de fonction au sein d'une `class` :

```
class Greeter {
    message: string;

    constructor(message: string) {
        this.message = message;
    }

    // surcharge
    sayHi(name: string): string;
    sayHi(names: string[]): ReadonlyArray<string>;

    // implémentation
    sayHi(name: unknown): unknown {
        if (typeof name === 'string') {
            return `${this.message}, ${name}!`;
        } else if (Array.isArray(name)) {
                    return  name.map(name  =>  `${this.message},
${name}!`);
        }
        throw new Error('value is invalid');
    }
}
console.log(new Greeter('Hello').sayHi('Simon'));
```

Fusion et extension

La fusion et l'extension désignent deux concepts différents liés à l'utilisation des types et des interfaces.

La fusion permet de combiner plusieurs déclarations portant le même nom en une seule définition, par exemple lorsque vous définissez plusieurs fois une interface portant le même nom :

```
interface X {
    a: string;
}
```

```
interface X {
    b: number;
}

const person: X = {
    a: 'a',
    b: 7,
};
```

L'extension désigne la possibilité d'étendre des types ou interfaces existants, ou d'en hériter, afin d'en créer de nouveaux. Il s'agit d'un mécanisme permettant d'ajouter des propriétés ou des méthodes supplémentaires à un type existant sans modifier sa définition d'origine. Exemple :

```
interface Animal {
    name: string;
    eat(): void;
}

interface Bird extends Animal {
    sing(): void;
}

const dog: Bird = {
    name: 'Bird 1',
    eat() {
        console.log('Eating');
    },
    sing() {
        console.log('Singing');
    },
};
```

Différences entre type et interface

Fusion de déclarations (augmentation) :

Les interfaces prennent en charge la fusion de déclarations, ce qui signifie que vous pouvez définir plusieurs interfaces portant le même nom et que TypeScript les fusionnera en une seule interface regroupant leurs propriétés et méthodes. En revanche, les types ne prennent pas en charge la fusion de déclarations. Cela peut être utile lorsque vous souhaitez ajouter des fonctionnalités ou personnaliser des types existants sans modifier les définitions d'origine, ou encore corriger des types manquants ou incorrects.

```
interface A {
    x: string;
}
interface A {
    y: string;
}
const j: A = {
    x: 'xx',
    y: 'yy',
};
```

Extension d'autres types/interfaces :

Les types comme les interfaces peuvent étendre d'autres types/interfaces, mais la syntaxe diffère. Avec les interfaces, vous utilisez le mot-clé `extends` pour hériter des propriétés et des méthodes d'autres interfaces. Toutefois, une interface ne peut pas étendre un type complexe tel qu'un type union.

```
interface A {
    x: string;
    y: number;
}
interface B extends A {
    z: string;
}
```

```
const car: B = {
    x: 'x',
    y: 123,
    z: 'z',
};
```

Pour les types, vous utilisez l'opérateur & afin de combiner plusieurs types en un seul type (intersection).

```
interface A {
    x: string;
    y: number;
}

type B = A & {
    j: string;
};

const c: B = {
    x: 'x',
    y: 123,
    j: 'j',
};
```

Types union et intersection :

Les types offrent davantage de souplesse pour définir des types union et intersection. Avec le mot-clé `type`, vous pouvez facilement créer des types union à l'aide de l'opérateur `|` et des types intersection à l'aide de l'opérateur `&`. Bien que les interfaces puissent également représenter indirectement des types union, elles ne prennent pas en charge nativement les types intersection.

```
type Department = 'dep-x' | 'dep-y'; // Union

type Person = {
```

```
    name: string;
    age: number;
};

type Employee = {
    id: number;
    department: Department;
};

type EmployeeInfo = Person & Employee; // Intersection
```

Exemple avec des interfaces :

```
interface A {
    x: 'x';
}
interface B {
    y: 'y';
}

type C = A | B; // Union d'interfaces
```

Classe

Syntaxe courante d'une classe

Le mot-clé `class` sert à définir une classe en TypeScript. Vous trouverez un exemple ci-dessous :

```
class Person {
    private name: string;
    private age: number;
    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }
    public sayHi(): void {
```

```
        console.log(
                    `Hello, my name is ${this.name} and I am
${this.age} years old.`
        );
    }
}
```

Le mot-clé `class` sert à définir une classe nommée « Person ».

La classe possède deux propriétés privées : name de type `string` et age de type `number`.

Le constructeur est défini à l'aide du mot-clé `constructor`. Il reçoit name et age en paramètres et les affecte aux propriétés correspondantes.

La classe possède une méthode `public` nommée sayHi qui affiche un message de salutation.

Pour créer une instance d'une classe en TypeScript, vous pouvez utiliser le mot-clé `new` suivi du nom de la classe, puis de parenthèses `()`. Par exemple :

```
const myObject = new Person('John Doe', 25);
myObject.sayHi(); // Sortie : Hello, my name is John Doe and I
am 25 years old.
```

Constructeur

Les constructeurs sont des méthodes spéciales au sein d'une classe. Ils permettent d'initialiser les propriétés de l'objet lorsqu'une instance de la classe est créée.

```
class Person {
    public name: string;
    public age: number;
```

```
    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }

    sayHello() {
        console.log(
            `Hello, my name is ${this.name} and I'm ${this.age}
years old.`
        );
    }
}

const john = new Person('Simon', 17);
john.sayHello();
```

Il est possible de surcharger un constructeur à l'aide de la syntaxe suivante :

```
type Sex = 'm' | 'f';

class Person {
    name: string;
    age: number;
    sex: Sex;

    constructor(name: string, age: number, sex?: Sex);
    constructor(name: string, age: number, sex: Sex) {
        this.name = name;
        this.age = age;
        this.sex = sex ?? 'm';
    }
}

const p1 = new Person('Simon', 17);
const p2 = new Person('Alice', 22, 'f');
```

En TypeScript, il est possible de définir plusieurs surcharges de constructeur, mais vous ne pouvez disposer que d'une seule implémentation, qui doit être compatible avec toutes les surcharges. Pour cela, vous pouvez utiliser un paramètre facultatif.

```
class Person {
    name: string;
    age: number;

    constructor();
    constructor(name: string);
    constructor(name: string, age: number);
    constructor(name?: string, age?: number) {
        this.name = name ?? 'Unknown';
        this.age = age ?? 0;
    }

    displayInfo() {
        console.log(`Name: ${this.name}, Age: ${this.age}`);
    }
}

const person1 = new Person();
person1.displayInfo(); // Name: Unknown, Age: 0

const person2 = new Person('John');
person2.displayInfo(); // Name: John, Age: 0

const person3 = new Person('Jane', 25);
person3.displayInfo(); // Name: Jane, Age: 25
```

Constructeurs privés et protégés

En TypeScript, les constructeurs peuvent être déclarés privés ou protégés, ce qui restreint leur accessibilité et leur utilisation.

Constructeurs privés : Ils peuvent uniquement être appelés depuis la classe elle-même. Les constructeurs privés sont souvent utilisés lorsque vous souhaitez imposer un patron de conception singleton ou limiter la création d'instances à une méthode de fabrique au sein de la classe.

Constructeurs protégés : Les constructeurs protégés sont utiles lorsque vous souhaitez créer une classe de base qui ne doit pas être instanciée directement, mais qui peut être étendue par des sous-classes.

```
class BaseClass {
    protected constructor() {}
}

class DerivedClass extends BaseClass {
    private value: number;

    constructor(value: number) {
        super();
        this.value = value;
    }
}

// Tenter d'instancier directement la classe de base provoquera une erreur
// const baseObj = new BaseClass(); // Erreur : le constructeur de la classe 'BaseClass' est protégé.

// Créer une instance de la classe dérivée
const derivedObj = new DerivedClass(10);
```

Modificateurs d'accès

Les modificateurs d'accès `private`, `protected` et `public` servent à contrôler la visibilité et l'accessibilité des membres d'une classe,

tels que les propriétés et les méthodes, dans les classes TypeScript. Ces modificateurs sont essentiels pour garantir l'encapsulation et établir des limites pour l'accès à l'état interne d'une classe et sa modification.

Le modificateur `private` restreint l'accès au membre de la classe à la classe qui le contient.

Le modificateur `protected` autorise l'accès au membre de la classe depuis la classe qui le contient et ses classes dérivées.

Le modificateur `public` accorde un accès sans restriction au membre de la classe, ce qui permet d'y accéder depuis n'importe où.

Accesseurs get et set

Les accesseurs et mutateurs sont des méthodes spéciales qui permettent de définir un comportement personnalisé pour l'accès aux propriétés d'une classe et leur modification. Ils permettent d'encapsuler l'état interne d'un objet et d'ajouter une logique supplémentaire lors de la lecture ou de la définition des valeurs des propriétés. En TypeScript, les accesseurs et les mutateurs sont définis respectivement à l'aide des mots-clés `get` et `set`. Voici un exemple :

```
class MyClass {
    private _myProperty: string;

    constructor(value: string) {
        this._myProperty = value;
    }
    get myProperty(): string {
        return this._myProperty;
```

```
    }
    set myProperty(value: string) {
        this._myProperty = value;
    }
}
```

Auto-accesseurs dans les classes

La version 4.9 de TypeScript ajoute la prise en charge des auto-accesseurs, une fonctionnalité ECMAScript à venir. Ils ressemblent aux propriétés de classe, mais sont déclarés avec le mot-clé « accessor ».

```
class Animal {
    accessor name: string;

    constructor(name: string) {
        this.name = name;
    }
}
```

Les auto-accesseurs sont « désucrés » en accesseurs `get` et `set` privés, opérant sur une propriété inaccessible.

```
class Animal {
    #__name: string;

    get name() {
        return this.#__name;
    }
    set name(value: string) {
        this.#__name = value;
    }

    constructor(name: string) {
        this.name = name;
```

```
    }
}
```

this

En TypeScript, le mot-clé `this` fait référence à l'instance actuelle d'une classe au sein de ses méthodes ou de ses constructeurs. Il permet d'accéder aux propriétés et aux méthodes de la classe et de les modifier depuis sa propre portée. Il fournit un moyen d'accéder à l'état interne d'un objet et de le manipuler au sein de ses propres méthodes.

```
class Person {
    private name: string;
    constructor(name: string) {
        this.name = name;
    }
    public introduce(): void {
        console.log(`Hello, my name is ${this.name}.`);
    }
}

const person1 = new Person('Alice');
person1.introduce(); // Hello, my name is Alice.
```

Propriétés de paramètres

Les propriétés de paramètres permettent de déclarer et d'initialiser les propriétés d'une classe directement dans les paramètres du constructeur, ce qui évite le code répétitif. Par exemple :

```
class Person {
    constructor(
        private name: string,
        public age: number
```

```
    ) {
            // Les mots-clés "private" et "public" dans le
constructeur
            // déclarent et initialisent automatiquement les
propriétés correspondantes de la classe.
    }
    public introduce(): void {
        console.log(
                    `Hello, my name is ${this.name} and I am
${this.age} years old.`
        );
    }
}
const person = new Person('Alice', 25);
person.introduce();
```

Classes abstraites

Les classes abstraites sont principalement utilisées en TypeScript pour l'héritage. Elles permettent de définir des propriétés et des méthodes communes dont les sous-classes peuvent hériter. C'est utile lorsque vous souhaitez définir un comportement commun et imposer aux sous-classes d'implémenter certaines méthodes. Elles permettent de créer une hiérarchie de classes dans laquelle la classe de base abstraite fournit une interface partagée et des fonctionnalités communes aux sous-classes.

```
abstract class Animal {
    protected name: string;

    constructor(name: string) {
        this.name = name;
    }

    abstract makeSound(): void;
}
```

```
class Cat extends Animal {
    makeSound(): void {
        console.log(`${this.name} meows.`);
    }
}

const cat = new Cat('Whiskers');
cat.makeSound(); // Sortie : Whiskers meows.
```

Avec des génériques

Les classes avec des génériques permettent de définir des classes réutilisables pouvant fonctionner avec différents types.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }

    setItem(item: T): void {
        this.item = item;
    }
}

const container1 = new Container<number>(42);
console.log(container1.getItem()); //  42

const container2 = new Container<string>('Hello');
container2.setItem('World');
console.log(container2.getItem()); // World
```

Décorateurs

Les décorateurs fournissent un mécanisme permettant d'ajouter des métadonnées, de modifier le comportement, de valider ou d'étendre les fonctionnalités de l'élément cible. Ces fonctions sont invoquées lors de l'exécution du programme. Plusieurs décorateurs peuvent être appliqués à une déclaration.

Les décorateurs sont des fonctionnalités expérimentales, et les exemples suivants sont uniquement compatibles avec TypeScript version 5 ou ultérieure avec ES6.

Pour les versions de TypeScript antérieures à la version 5, ils doivent être activés à l'aide de la propriété `experimentalDecorators` dans votre fichier `tsconfig.json` ou en utilisant `--experimentalDecorators` dans votre ligne de commande (mais l'exemple suivant ne fonctionnera pas).

Voici quelques cas d'utilisation courants des décorateurs :

- Surveillance des modifications de propriétés.
- Surveillance des appels de méthodes.
- Ajout de propriétés ou de méthodes supplémentaires.
- Validation à l'exécution.
- Sérialisation et désérialisation automatiques.
- Journalisation.
- Autorisation et authentification.
- Protection contre les erreurs.

Remarque : les décorateurs de la version 5 ne permettent pas de décorer les paramètres.

Types de décorateurs :

Décorateurs de classe

Les décorateurs de classe sont utiles pour étendre une classe existante, par exemple en ajoutant des propriétés ou des méthodes, ou en collectant des instances d'une classe. Dans l'exemple suivant, nous ajoutons une méthode `toString` qui convertit la classe en une représentation sous forme de chaîne.

```
type Constructor<T = {}> = new (...args: any[]) => T;

function toString<Class extends Constructor>(
    Value: Class,
    context: ClassDecoratorContext<Class>
) {
    return class extends Value {
        constructor(...args: any[]) {
            super(...args);
            console.log(JSON.stringify(this));
            console.log(JSON.stringify(context));
        }
    };
}

@toString
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    greet() {
        return 'Hello, ' + this.name;
    }
}
const person = new Person('Simon');
/* Affiche :
```

```
{"name":"Simon"}
{"kind":"class","name":"Person"}
*/
```

Décorateur de propriété

Les décorateurs de propriété sont utiles pour modifier le comportement d'une propriété, par exemple en changeant les valeurs d'initialisation. Dans le code suivant, nous avons un script qui définit une propriété pour qu'elle soit toujours en majuscules :

```
function upperCase<T>(
    target: undefined,
    context: ClassFieldDecoratorContext<T, string>
) {
    return function (this: T, value: string) {
        return value.toUpperCase();
    };
}

class MyClass {
    @upperCase
    prop1 = 'hello!';
}

console.log(new MyClass().prop1); // Affiche : HELLO!
```

Décorateur de méthode

Les décorateurs de méthode permettent de modifier ou d'améliorer le comportement des méthodes. Voici un exemple simple de journalisation :

```
function log<This, Args extends any[], Return>(
    target: (this: This, ...args: Args) => Return,
```

```
    context: ClassMethodDecoratorContext<
        This,
        (this: This, ...args: Args) => Return
    >
) {
    const methodName = String(context.name);

       function replacementMethod(this: This, ...args: Args):
Return {
        console.log(`LOG: Entering method '${methodName}'.`);
        const result = target.call(this, ...args);
        console.log(`LOG: Exiting method '${methodName}'.`);
        return result;
    }

    return replacementMethod;
}

class MyClass {
    @log
    sayHello() {
        console.log('Hello!');
    }
}

new MyClass().sayHello();
```

Il affiche :

```
LOG: Entering method 'sayHello'.
Hello!
LOG: Exiting method 'sayHello'.
```

Décorateurs d'accesseurs et de mutateurs

Les décorateurs d'accesseurs et de mutateurs permettent de modifier ou d'améliorer le comportement des accesseurs de

classe. Ils sont utiles, par exemple, pour valider les affectations de propriétés. Voici un exemple simple de décorateur d'accesseur :

```
function range<This, Return extends number>(min: number, max:
number) {
    return function (
        target: (this: This) => Return,
        context: ClassGetterDecoratorContext<This, Return>
    ) {
        return function (this: This): Return {
            const value = target.call(this);
            if (value < min || value > max) {
                throw 'Invalid';
            }
            Object.defineProperty(this, context.name, {
                value,
                enumerable: true,
            });
            return value;
        };
    };
}

class MyClass {
    private _value = 0;

    constructor(value: number) {
        this._value = value;
    }
    @range(1, 100)
    get getValue(): number {
        return this._value;
    }
}

const obj = new MyClass(10);
console.log(obj.getValue); // Valide : 10
```

```
const obj2 = new MyClass(999);
console.log(obj2.getValue); // Lève une exception : Invalid!
```

Métadonnées des décorateurs

Les métadonnées des décorateurs simplifient le processus permettant aux décorateurs d'appliquer et d'utiliser des métadonnées dans n'importe quelle classe. Ils peuvent accéder à une nouvelle propriété de métadonnées sur l'objet de contexte, qui peut servir de clé aussi bien pour les valeurs primitives que pour les objets. Les informations de métadonnées sont accessibles sur la classe via `Symbol.metadata`.

Les métadonnées peuvent être utilisées à diverses fins, telles que le débogage, la sérialisation ou l'injection de dépendances avec des décorateurs.

```
//@ts-ignore
Symbol.metadata ??= Symbol('Symbol.metadata'); // Polyfill simple

type Context =
    | ClassFieldDecoratorContext
    | ClassAccessorDecoratorContext
     | ClassMethodDecoratorContext; // Le contexte contient les métadonnées de propriété : DecoratorMetadata

function setMetadata(_target: any, context: Context) {
    // Définir l'objet de métadonnées avec une valeur primitive
    context.metadata[context.name] = true;
}

class MyClass {
    @setMetadata
    a = 123;
```

```
    @setMetadata
    accessor b = 'b';

    @setMetadata
    fn() {}
}

const metadata = MyClass[Symbol.metadata]; // Obtenir les
métadonnées

console.log(JSON.stringify(metadata)); //
{"bar":true,"baz":true,"foo":true}
```

Héritage

L'héritage désigne le mécanisme par lequel une classe peut hériter des propriétés et des méthodes d'une autre classe, appelée classe de base ou superclasse. La classe dérivée, également appelée classe enfant ou sous-classe, peut étendre et spécialiser les fonctionnalités de la classe de base en ajoutant de nouvelles propriétés et méthodes ou en redéfinissant celles qui existent.

```
class Animal {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    speak(): void {
        console.log('The animal makes a sound');
    }
}

class Dog extends Animal {
```

```
    breed: string;

    constructor(name: string, breed: string) {
        super(name);
        this.breed = breed;
    }

    speak(): void {
        console.log('Woof! Woof!');
    }
}

// Créer une instance de la classe de base
const animal = new Animal('Generic Animal');
animal.speak(); // The animal makes a sound

// Créer une instance de la classe dérivée
const dog = new Dog('Max', 'Labrador');
dog.speak(); // Woof! Woof!"
```

TypeScript ne prend pas en charge l'héritage multiple au sens traditionnel et permet à la place l'héritage à partir d'une seule classe de base. TypeScript prend en charge plusieurs interfaces. Une interface peut définir un contrat pour la structure d'un objet, et une classe peut implémenter plusieurs interfaces. Cela permet à une classe d'hériter du comportement et de la structure de plusieurs sources.

```
interface Flyable {
    fly(): void;
}

interface Swimmable {
    swim(): void;
}

class FlyingFish implements Flyable, Swimmable {
```

```
    fly() {
        console.log('Flying...');
    }

    swim() {
        console.log('Swimming...');
    }
}

const flyingFish = new FlyingFish();
flyingFish.fly();
flyingFish.swim();
```

Le mot-clé `class` en TypeScript, comme en JavaScript, est souvent qualifié de sucre syntaxique. Il a été introduit dans ECMAScript 2015 (ES6) afin d'offrir une syntaxe plus familière pour créer et manipuler des objets selon un modèle fondé sur les classes. Il est toutefois important de noter que TypeScript, étant un sur-ensemble de JavaScript, est finalement compilé en JavaScript, qui reste fondamentalement fondé sur les prototypes.

Membres statiques

TypeScript possède des membres statiques. Pour accéder aux membres statiques d'une classe, vous pouvez utiliser le nom de la classe suivi d'un point, sans avoir à créer d'objet.

```
class OfficeWorker {
    static memberCount: number = 0;

    constructor(private name: string) {
        OfficeWorker.memberCount++;
    }
}
```

```
const w1 = new OfficeWorker('James');
const w2 = new OfficeWorker('Simon');
const total = OfficeWorker.memberCount;
console.log(total); // 2
```

Initialisation des propriétés

Il existe plusieurs façons d'initialiser les propriétés d'une classe en TypeScript :

En ligne :

Dans l'exemple suivant, ces valeurs initiales seront utilisées lors de la création d'une instance de la classe.

```
class MyClass {
    property1: string = 'default value';
    property2: number = 42;
}
```

Dans le constructeur :

```
class MyClass {
    property1: string;
    property2: number;

    constructor() {
        this.property1 = 'default value';
        this.property2 = 42;
    }
}
```

À l'aide des paramètres du constructeur :

```
class MyClass {
    constructor(
        private property1: string = 'default value',
```

```
        public property2: number = 42
    ) {
        // Il n'est pas nécessaire d'affecter explicitement les
valeurs aux propriétés.
    }
    log() {
        console.log(this.property2);
    }
}
const x = new MyClass();
x.log();
```

Surcharge de méthodes

La surcharge de méthodes permet à une classe de disposer de plusieurs méthodes portant le même nom, mais avec des types de paramètres différents ou un nombre de paramètres différent. Cela permet d'appeler une méthode de différentes manières selon les arguments transmis.

```
class MyClass {
       add(a: number, b: number): number; // Signature de
surcharge 1
       add(a: string, b: string): string; // Signature de
surcharge 2

      add(a: number | string, b: number | string): number |
string {
        if (typeof a === 'number' && typeof b === 'number') {
            return a + b;
        }
        if (typeof a === 'string' && typeof b === 'string') {
            return a.concat(b);
        }
        throw new Error('Invalid arguments');
    }
}
```

```
const r = new MyClass();
console.log(r.add(10, 5)); // Affiche 15
```

Génériques

Les génériques permettent de créer des composants et des fonctions réutilisables pouvant fonctionner avec plusieurs types. Grâce aux génériques, vous pouvez paramétrer des types, des fonctions et des interfaces, ce qui leur permet d'opérer sur différents types sans les spécifier explicitement au préalable.

Les génériques permettent de rendre le code plus flexible et réutilisable.

Type générique

Pour définir un type générique, vous utilisez des chevrons (<>) afin de spécifier les paramètres de type, par exemple :

```
function identity<T>(arg: T): T {
    return arg;
}
const a = identity('x');
const b = identity(123);

const getLen = <T,>(data: ReadonlyArray<T>) => data.length;
const len = getLen([1, 2, 3]);
```

Classes génériques

Les génériques peuvent également être appliqués aux classes. Celles-ci peuvent ainsi fonctionner avec plusieurs types à l'aide de paramètres de type. C'est utile pour créer des définitions de

classes réutilisables capables d'opérer sur différents types de données tout en préservant la sécurité du typage.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }
}

const numberContainer = new Container<number>(123);
console.log(numberContainer.getItem()); // 123

const stringContainer = new Container<string>('hello');
console.log(stringContainer.getItem()); // hello
```

Contraintes génériques

Les paramètres génériques peuvent être contraints à l'aide du mot-clé `extends`, suivi d'un type ou d'une interface que le paramètre de type doit respecter.

Dans l'exemple suivant, `T` doit posséder une propriété `length` correctement typée pour être valide :

```
const printLen = <T extends { length: number }>(value: T): void
=> {
    console.log(value.length);
};

printLen('Hello'); // 5
printLen([1, 2, 3]); // 3
```

```
printLen({ length: 10 }); // 10
printLen(123); // Invalide
```

Une fonctionnalité générique notable introduite dans la version 3.4 RC est l'inférence de type des fonctions d'ordre supérieur, qui propage les arguments de type générique :

```
declare function pipe<A extends any[], B, C>(
    ab: (...args: A) => B,
    bc: (b: B) => C
): (...args: A) => C;

declare function list<T>(a: T): T[];
declare function box<V>(x: V): { value: V };

const listBox = pipe(list, box); // <T>(a: T) => { value: T[] }
const boxList = pipe(box, list); // <V>(x: V) => { value: V }[]
```

Cette fonctionnalité facilite la programmation avec un typage sûr dans un style point-free, courant en programmation fonctionnelle.

Réduction contextuelle des génériques

La réduction contextuelle des génériques est le mécanisme de TypeScript qui permet au compilateur de restreindre le type d'un paramètre générique en fonction du contexte dans lequel il est utilisé. Elle est utile lorsque des types génériques sont employés dans des instructions conditionnelles :

```
function process<T>(value: T): void {
    if (typeof value === 'string') {
        // La valeur est affinée au type 'string'
        console.log(value.length);
    } else if (typeof value === 'number') {
        // La valeur est affinée au type 'number'
```

```
        console.log(value.toFixed(2));
    }
}

process('hello'); // 5
process(3.14159); // 3.14
```

Types structurels effacés

En TypeScript, les objets ne sont pas tenus de correspondre à un type spécifique et exact. Par exemple, si nous créons un objet qui satisfait aux exigences d'une interface, nous pouvons utiliser cet objet aux endroits où cette interface est requise, même s'il n'existe aucun lien explicite entre eux. Exemple :

```
type NameProp1 = {
    prop1: string;
};

function log(x: NameProp1) {
    console.log(x.prop1);
}

const obj = {
    prop2: 123,
    prop1: 'Origin',
};

log(obj); // Valide
```

Espaces de noms

En TypeScript, les espaces de noms servent à organiser le code dans des conteneurs logiques, afin d'éviter les collisions de noms et de regrouper le code associé. L'utilisation du mot-clé

`export` permet d'accéder à l'espace de noms depuis l'extérieur des modules.

```
export namespace MyNamespace {
    export interface MyInterface1 {
        prop1: boolean;
    }
    export interface MyInterface2 {
        prop2: string;
    }
}

const a: MyNamespace.MyInterface1 = {
    prop1: true,
};
```

Symboles

Les symboles sont un type de données primitif qui représente une valeur immuable dont l'unicité globale est garantie pendant toute la durée de vie du programme.

Les symboles peuvent être utilisés comme clés de propriétés d'objet et permettent de créer des propriétés non énumérables.

```
const key1: symbol = Symbol('key1');
const key2: symbol = Symbol('key2');

const obj = {
    [key1]: 'value 1',
    [key2]: 'value 2',
};

console.log(obj[key1]); // value 1
console.log(obj[key2]); // value 2
```

Dans les WeakMaps et les WeakSets, les symboles peuvent désormais servir de clés.

Directives à triple barre oblique

Les directives à triple barre oblique sont des commentaires spéciaux qui indiquent au compilateur comment traiter un fichier. Ces directives commencent par trois barres obliques consécutives (`///`), sont généralement placées en haut d'un fichier TypeScript et n'ont aucun effet sur le comportement à l'exécution.

Les directives à triple barre oblique servent à référencer des dépendances externes, à spécifier le comportement de chargement des modules, à activer ou désactiver certaines fonctionnalités du compilateur, et plus encore. Voici quelques exemples :

Référencement d'un fichier de déclaration :

```
/// <reference path="path/to/declaration/file.d.ts" />
```

Indication du format du module :

```
/// <amd|commonjs|system|umd|es6|es2015|none>
```

Activation des options du compilateur, dans l'exemple suivant le mode strict :

```
/// <strict|noImplicitAny|noUnusedLocals|noUnusedParameters>
```

Manipulation des types

Création de types à partir de types

Il est possible de créer de nouveaux types en composant, manipulant ou transformant des types existants.

Types d'intersection (`&`) :

Permettent de combiner plusieurs types en un seul type :

```
type A = { foo: number };
type B = { bar: string };
type C = A & B; // Intersection de A et B
const obj: C = { foo: 42, bar: 'hello' };
```

Types d'union (`|`) :

Permettent de définir un type qui peut être l'un de plusieurs types :

```
type Result = string | number;
const value1: Result = 'hello';
const value2: Result = 42;
```

Types mappés :

Permettent de transformer les propriétés d'un type existant afin de créer un nouveau type :

```
type Mutable<T> = {
    readonly [P in keyof T]: T[P];
};
type Person = {
    name: string;
    age: number;
};
type ImmutablePerson = Mutable<Person>; // Les propriétés deviennent en lecture seule
```

Types conditionnels :

Permettent de créer des types en fonction de certaines conditions :

```
type ExtractParam<T> = T extends (param: infer P) => any ? P :
never;
type MyFunction = (name: string) => number;
type ParamType = ExtractParam<MyFunction>; // string
```

Types d'accès indexé

En TypeScript, il est possible d'accéder aux types des propriétés d'un autre type et de les manipuler à l'aide d'un index, `Type[Key]`.

```
type Person = {
    name: string;
    age: number;
};

type AgeType = Person['age']; // number

type MyTuple = [string, number, boolean];
type MyType = MyTuple[2]; // boolean
```

Types utilitaires

Plusieurs types utilitaires intégrés peuvent être utilisés pour manipuler les types. Voici la liste des plus couramment utilisés :

Awaited<T>

Construit un type qui désencapsule récursivement les types Promise.

```
type A = Awaited<Promise<string>>; // string
```

Partial<T>

Construit un type dont toutes les propriétés de T sont définies comme facultatives.

```
type Person = {
    name: string;
    age: number;
};

type A = Partial<Person>; // { name?: string | undefined; age?: number | undefined; }
```

Required<T>

Construit un type dont toutes les propriétés de T sont définies comme obligatoires.

```
type Person = {
    name?: string;
    age?: number;
};

type A = Required<Person>; // { name: string; age: number; }
```

Readonly<T>

Construit un type dont toutes les propriétés de T sont définies en lecture seule.

```
type Person = {
    name: string;
    age: number;
};

type A = Readonly<Person>;
```

```ts
const a: A = { name: 'Simon', age: 17 };
a.name = 'John'; // Invalide
```

Record<K, T>

Construit un type avec un ensemble de propriétés K de type T.

```ts
type Product = {
    name: string;
    price: number;
};

const products: Record<string, Product> = {
    apple: { name: 'Apple', price: 0.5 },
    banana: { name: 'Banana', price: 0.25 },
};

console.log(products.apple); // { name: 'Apple', price: 0.5 }
```

Pick<T, K>

Construit un type en sélectionnant dans T les propriétés K spécifiées.

```ts
type Product = {
    name: string;
    price: number;
};

type Price = Pick<Product, 'price'>; // { price: number; }
```

Omit<T, K>

Construit un type en omettant dans T les propriétés K spécifiées.

```ts
type Product = {
    name: string;
    price: number;
};

type Name = Omit<Product, 'price'>; // { name: string; }
```

Exclude<T, U>

Construit un type en excluant de T toutes les valeurs de type U.

```ts
type Union = 'a' | 'b' | 'c';
type MyType = Exclude<Union, 'a' | 'c'>; // b
```

Extract<T, U>

Construit un type en extrayant de T toutes les valeurs de type U.

```ts
type Union = 'a' | 'b' | 'c';
type MyType = Extract<Union, 'a' | 'c'>; // a | c
```

NonNullable<T>

Construit un type en excluant null et undefined de T.

```ts
type Union = 'a' | null | undefined | 'b';
type MyType = NonNullable<Union>; // 'a' | 'b'
```

Parameters<T>

Extrait les types des paramètres d'un type de fonction T.

```ts
type Func = (a: string, b: number) => void;
type MyType = Parameters<Func>; // [a: string, b: number]
```

ConstructorParameters<T>

Extrait les types des paramètres d'un type de fonction constructeur T.

```
class Person {
    constructor(
        public name: string,
        public age: number
    ) {}
}
type PersonConstructorParams = ConstructorParameters<typeof
Person>; // [name: string, age: number]
const params: PersonConstructorParams = ['John', 30];
const person = new Person(...params);
console.log(person); // Person { name: 'John', age: 30 }
```

ReturnType<T>

Extrait le type de retour d'un type de fonction T.

```
type Func = (name: string) => number;
type MyType = ReturnType<Func>; // number
```

InstanceType<T>

Extrait le type d'instance d'un type de classe T.

```
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    sayHello() {
```

```
        console.log(`Hello, my name is ${this.name}!`);
    }
}

type PersonInstance = InstanceType<typeof Person>;

const person: PersonInstance = new Person('John');

person.sayHello(); // Hello, my name is John!
```

ThisParameterType<T>

Extrait le type du paramètre ‘this’ d’un type de fonction T.

```
interface Person {
    name: string;
    greet(this: Person): void;
}
type PersonThisType = ThisParameterType<Person['greet']>; // Person
```

OmitThisParameter<T>

Supprime le paramètre ‘this’ d’un type de fonction T.

```
function capitalize(this: String) {
                        return this[0].toUpperCase() +
this.substring(1).toLowerCase();
}

type CapitalizeType = OmitThisParameter<typeof capitalize>; // () => string
```

ThisType<T>

Sert de marqueur pour un type `this` contextuel.

```
type Logger = {
    log: (error: string) => void;
};

let helperFunctions: { [name: string]: Function } &
ThisType<Logger> = {
    hello: function () {
            this.log('some error'); // Valide, car "log" fait
partie de "this".
        this.update(); // Invalide
    },
};
```

Uppercase<T>

Met en majuscules le nom du type d'entrée T.

```
type MyType = Uppercase<'abc'>; // "ABC"
```

Lowercase<T>

Met en minuscules le nom du type d'entrée T.

```
type MyType = Lowercase<'ABC'>; // "abc"
```

Capitalize<T>

Met en majuscule la première lettre du nom du type d'entrée T.

```
type MyType = Capitalize<'abc'>; // "Abc"
```

Uncapitalize<T>

Met en minuscule la première lettre du nom du type d'entrée T.

```
type MyType = Uncapitalize<'Abc'>; // "abc"
```

NoInfer<T>

NoInfer est un type utilitaire conçu pour bloquer l'inférence automatique des types dans la portée d'une fonction générique.

Exemple :

```
// Inférence automatique des types dans la portée d'une fonction générique.
function fn<T extends string>(x: T[], y: T) {
    return x.concat(y);
}
const r = fn(['a', 'b'], 'c'); // Le type est ici ("a" | "b" | "c")[]
```

Avec NoInfer :

```
// Exemple de fonction utilisant NoInfer pour empêcher l'inférence de type
function fn2<T extends string>(x: T[], y: NoInfer<T>) {
    return x.concat(y);
}

const r2 = fn2(['a', 'b'], 'c'); // Erreur : un argument de type '"c"' n'est pas assignable à un paramètre de type '"a" | "b"'.
```

Autres

Gestion des erreurs et des exceptions

TypeScript permet d'intercepter et de gérer les erreurs à l'aide des mécanismes standard de gestion des erreurs de JavaScript :

Blocs try-catch-finally :

```
try {
    // Code susceptible de lever une erreur
} catch (error) {
    // Gérer l'erreur
} finally {
    // Code toujours exécuté, finally est facultatif
}
```

Vous pouvez également gérer différents types d'erreurs :

```
try {
    // Code susceptible de lever différents types d'erreurs
} catch (error) {
    if (error instanceof TypeError) {
        // Gérer TypeError
    } else if (error instanceof RangeError) {
        // Gérer RangeError
    } else {
        // Gérer les autres erreurs
    }
}
```

Types d'erreurs personnalisés :

Il est possible de spécifier des erreurs plus précises en étendant la `class` Error :

```
class CustomError extends Error {
    constructor(message: string) {
        super(message);
        this.name = 'CustomError';
    }
}

throw new CustomError('This is a custom error.');
```

Classes mixin

Les classes mixin permettent de combiner et de composer le comportement de plusieurs classes dans une seule classe. Elles permettent de réutiliser et d'étendre des fonctionnalités sans recourir à de profondes chaînes d'héritage.

```
abstract class Identifiable {
    name: string = '';
    logId() {
        console.log('id:', this.name);
    }
}
abstract class Selectable {
    selected: boolean = false;
    select() {
        this.selected = true;
        console.log('Select');
    }
    deselect() {
        this.selected = false;
        console.log('Deselect');
    }
}
class MyClass {
    constructor() {}
}

// Étendre MyClass pour inclure le comportement de Identifiable
et Selectable
interface MyClass extends Identifiable, Selectable {}

// Fonction appliquant des mixins à une classe
function applyMixins(source: any, baseCtors: any[]) {
    baseCtors.forEach(baseCtor => {

Object.getOwnPropertyNames(baseCtor.prototype).forEach(name   =>
{
            let descriptor = Object.getOwnPropertyDescriptor(
                baseCtor.prototype,
```

```
                name
            );
            if (descriptor) {
                Object.defineProperty(source.prototype, name,
descriptor);
            }
        });
    });
}

// Appliquer les mixins à MyClass
applyMixins(MyClass, [Identifiable, Selectable]);
let o = new MyClass();
o.name = 'abc';
o.logId();
o.select();
```

Fonctionnalités asynchrones du langage

TypeScript étant un sur-ensemble de JavaScript, il intègre les fonctionnalités asynchrones du langage JavaScript telles que :

Promesses :

Les promesses permettent de gérer les opérations asynchrones et leurs résultats à l'aide de méthodes telles que `.then()` et `.catch()` pour traiter les cas de réussite et d'erreur.

Pour en savoir plus : https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

Async/await :

Les mots-clés async/await permettent d'utiliser une syntaxe d'apparence plus synchrone pour travailler avec les promesses. Le mot-clé `async` sert à définir une fonction asynchrone, et le

mot-clé `await` est utilisé au sein d'une fonction asynchrone pour suspendre l'exécution jusqu'à ce qu'une promesse soit résolue ou rejetée.

Pour en savoir plus : https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await

Les API suivantes sont bien prises en charge par TypeScript :

API Fetch : https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Web Workers : https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API

Shared Workers : https://developer.mozilla.org/en-US/docs/Web/API/SharedWorker

WebSocket : https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

Itérateurs et générateurs

Les itérateurs et les générateurs sont tous deux bien pris en charge par TypeScript.

Les itérateurs sont des objets qui implémentent le protocole d'itération, permettant d'accéder un à un aux éléments d'une collection ou d'une séquence. Il s'agit d'une structure qui contient un pointeur vers l'élément suivant de l'itération. Ils disposent d'une méthode `next()` qui renvoie la valeur suivante

de la séquence ainsi qu'un booléen indiquant si la séquence est done.

```
class NumberIterator implements Iterable<number> {
    private current: number;

    constructor(
        private start: number,
        private end: number
    ) {
        this.current = start;
    }

    public next(): IteratorResult<number> {
        if (this.current <= this.end) {
            const value = this.current;
            this.current++;
            return { value, done: false };
        } else {
            return { value: undefined, done: true };
        }
    }

    [Symbol.iterator](): Iterator<number> {
        return this;
    }
}

const iterator = new NumberIterator(1, 3);

for (const num of iterator) {
    console.log(num);
}
```

Les générateurs sont des fonctions spéciales définies à l'aide de la syntaxe `function*`, qui simplifie la création d'itérateurs. Ils utilisent le mot-clé `yield` pour définir la séquence de valeurs et

suspendent puis reprennent automatiquement l'exécution lorsque des valeurs sont demandées.

Les générateurs facilitent la création d'itérateurs et sont particulièrement utiles pour travailler avec des séquences volumineuses ou infinies.

Exemple :

```
function* numberGenerator(start: number, end: number): Generator<number> {
    for (let i = start; i <= end; i++) {
        yield i;
    }
}

const generator = numberGenerator(1, 5);

for (const num of generator) {
    console.log(num);
}
```

TypeScript prend également en charge les itérateurs asynchrones et les générateurs asynchrones.

Pour en savoir plus :

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Generator

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator

Référence TsDocs JSDoc

Lorsque vous travaillez avec une base de code JavaScript, vous pouvez aider TypeScript à inférer le bon type en utilisant des commentaires JSDoc accompagnés d'annotations supplémentaires pour fournir des informations de type.

Exemple :

```
/**
 * Calcule la puissance d'un nombre donné
 * @constructor
 * @param {number} base - La valeur de base de l'expression
  * @param {number} exponent - La valeur de l'exposant de l'expression
 */
function power(base: number, exponent: number) {
    return Math.pow(base, exponent);
}
power(10, 2); // function power(base: number, exponent: number): number
```

La documentation complète est disponible à ce lien : https://www.typescriptlang.org/docs/handbook/jsdoc-supported-types.html

Depuis la version 3.7, il est possible de générer des définitions de types .d.ts à partir de la syntaxe JSDoc de JavaScript. Vous trouverez plus d'informations ici : https://www.typescriptlang.org/docs/handbook/declaration-files/dts-from-js.html

@types

Les paquets de l'organisation @types suivent une convention de nommage spéciale et servent à fournir des définitions de types

pour les bibliothèques ou modules JavaScript existants. Par exemple, la commande :

```
npm install --save-dev @types/lodash
```

installera les définitions de types de `lodash` dans votre projet actuel.

Pour contribuer aux définitions de types d'un paquet `@types`, veuillez soumettre une pull request à https://github.com/DefinitelyTyped/DefinitelyTyped.

JSX

JSX (JavaScript XML) est une extension de la syntaxe du langage JavaScript qui vous permet d'écrire du code semblable à du HTML dans vos fichiers JavaScript ou TypeScript. Il est couramment utilisé dans React pour définir la structure HTML.

TypeScript étend les fonctionnalités de JSX en fournissant une vérification des types et une analyse statique.

Pour utiliser JSX, vous devez définir l'option de compilation `jsx` dans votre fichier `tsconfig.json`. Deux options de configuration courantes :

- "preserve" : génère des fichiers .jsx en laissant JSX inchangé. Cette option indique à TypeScript de conserver la syntaxe JSX telle quelle et de ne pas la transformer pendant le processus de compilation. Vous pouvez utiliser cette option si vous disposez d'un outil distinct, comme Babel, qui se charge de la transformation.
- "react" : active la transformation JSX intégrée à TypeScript. React.createElement sera utilisé.

Toutes les options sont disponibles ici : https://www.typescriptlang.org/tsconfig#jsx

Modules ES6

TypeScript prend en charge ES6 (ECMAScript 2015) ainsi que de nombreuses versions ultérieures. Cela signifie que vous pouvez utiliser la syntaxe ES6, comme les fonctions fléchées, les littéraux de gabarit, les classes, les modules, la déstructuration et bien plus encore.

Pour activer les fonctionnalités ES6 dans votre projet, vous pouvez spécifier la propriété `target` dans le tsconfig.json.

Exemple de configuration :

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "es6",
    "moduleResolution": "node",
    "sourceMap": true,
    "outDir": "dist"
  },
  "include": ["src"]
}
```

Opérateur d'exponentiation ES7

L'opérateur d'exponentiation (`**`) calcule la valeur obtenue en élevant le premier opérande à la puissance du second. Il fonctionne de manière similaire à `Math.pow()`, avec la possibilité supplémentaire d'accepter des BigInts comme opérandes. TypeScript prend entièrement en charge cet opérateur lorsque

`target` est défini sur `es2016` ou une version ultérieure dans votre fichier tsconfig.json.

```
console.log(2 ** (2 ** 2)); // 16
```

L'instruction for-await-of

Il s'agit d'une fonctionnalité JavaScript entièrement prise en charge dans TypeScript, qui vous permet d'itérer sur des objets itérables asynchrones avec la version cible `es2018`.

```
async function* asyncNumbers(): AsyncIterableIterator<number> {
    yield Promise.resolve(1);
    yield Promise.resolve(2);
    yield Promise.resolve(3);
}

(async () => {
    for await (const num of asyncNumbers()) {
        console.log(num);
    }
})();
```

Métapropriété new.target

Vous pouvez utiliser la métapropriété `new.target` dans TypeScript, qui vous permet de déterminer si une fonction ou un constructeur a été invoqué à l'aide de l'opérateur new. Elle permet de détecter si un objet a été créé à la suite d'un appel de constructeur.

```
class Parent {
    constructor() {
            console.log(new.target);  // Affiche la fonction
constructeur utilisée pour créer une instance
    }
```

```
}

class Child extends Parent {
    constructor() {
        super();
    }
}

const parentX = new Parent(); // [Function: Parent]
const child = new Child(); // [Function: Child]
```

Expressions d'importation dynamique

Il est possible de charger des modules de manière conditionnelle ou à la demande, grâce à la proposition ECMAScript d'importation dynamique, qui est prise en charge dans TypeScript.

La syntaxe des expressions d'importation dynamique dans TypeScript est la suivante :

```
async function renderWidget() {
    const container = document.getElementById('widget');
    if (container !== null) {
        const widget = await import('./widget'); // Importation
dynamique
        widget.render(container);
    }
}

renderWidget();
```

"tsc –watch"

Cette commande lance le compilateur TypeScript avec le paramètre `--watch`, ce qui lui permet de recompiler

automatiquement les fichiers TypeScript chaque fois qu'ils sont modifiés.

```
tsc --watch
```

Depuis TypeScript 4.9, la surveillance des fichiers repose principalement sur les événements du système de fichiers et revient automatiquement à l'interrogation périodique si aucun mécanisme de surveillance fondé sur les événements ne peut être établi.

Opérateur d'assertion non-null

L'opérateur d'assertion non-null (! postfixé), également appelé assertion d'affectation définie, est une fonctionnalité TypeScript qui vous permet d'affirmer qu'une variable ou une propriété n'est ni null ni undefined, même si l'analyse statique des types de TypeScript suggère qu'elle pourrait l'être. Cette fonctionnalité permet de supprimer toute vérification explicite.

```
type Person = {
    name: string;
};

const printName = (person?: Person) => {
    console.log(`Name is ${person!.name}`);
};
```

Déclarations avec valeurs par défaut

Les déclarations avec valeurs par défaut sont utilisées lorsqu'une valeur par défaut est affectée à une variable ou à un paramètre. Cela signifie que si aucune valeur n'est fournie pour

cette variable ou ce paramètre, la valeur par défaut sera utilisée à la place.

```
function greet(name: string = 'Anonymous'): void {
    console.log(`Hello, ${name}!`);
}
greet(); // Hello, Anonymous!
greet('John'); // Hello, John!
```

Chaînage optionnel

L'opérateur de chaînage optionnel `?.` fonctionne comme l'opérateur point habituel (`.`) pour accéder aux propriétés ou aux méthodes. Toutefois, il gère sans erreur les valeurs null ou undefined en interrompant l'expression et en renvoyant `undefined`, au lieu de lever une erreur.

```
type Person = {
    name: string;
    age?: number;
    address?: {
        street?: string;
        city?: string;
    };
};

const person: Person = {
    name: 'John',
};

console.log(person.address?.city); // undefined
```

Opérateur de coalescence des valeurs nulles

L'opérateur de coalescence des valeurs nulles `??` renvoie la valeur de droite si la valeur de gauche est `null` ou `undefined` ;

sinon, il renvoie la valeur de gauche.

```
const foo = null ?? 'foo';
console.log(foo); // foo

const baz = 1 ?? 'baz';
const baz2 = 0 ?? 'baz';
console.log(baz); // 1
console.log(baz2); // 0
```

Types littéraux de gabarit

Les types littéraux de gabarit vous permettent de manipuler des valeurs de chaîne au niveau des types et de générer de nouveaux types de chaîne à partir de types existants. Ils sont utiles pour créer des types plus expressifs et plus précis à partir d'opérations sur des chaînes.

```
type Department = 'engineering' | 'hr';
type Language = 'english' | 'spanish';
type Id = `${Department}-${Language}-id`; // "engineering-english-id" | "engineering-spanish-id" | "hr-english-id" | "hr-spanish-id"
```

Surcharge de fonctions

La surcharge de fonctions vous permet de définir plusieurs signatures de fonction pour un même nom de fonction, chacune avec des types de paramètres et de retour différents. Lorsque vous appelez une fonction surchargée, TypeScript utilise les arguments fournis pour déterminer la bonne signature de fonction :

```
function makeGreeting(name: string): string;
function makeGreeting(names: string[]): string[];
```

```
function makeGreeting(person: unknown): unknown {
    if (typeof person === 'string') {
        return `Hi ${person}!`;
    } else if (Array.isArray(person)) {
        return person.map(name => `Hi, ${name}!`);
    }
    throw new Error('Unable to greet');
}

makeGreeting('Simon');
makeGreeting(['Simone', 'John']);
```

Types récursifs

Un type récursif est un type qui peut faire référence à lui-même. Cela est utile pour définir des structures de données dotées d'une structure hiérarchique ou récursive (avec une imbrication potentiellement infinie), telles que les listes chaînées, les arbres et les graphes.

```
type ListNode<T> = {
    data: T;
    next: ListNode<T> | undefined;
};
```

Types conditionnels récursifs

Il est possible de définir des relations de types complexes à l'aide de la logique et de la récursivité dans TypeScript. Décomposons cela en termes simples :

Les types conditionnels vous permettent de définir des types en fonction de conditions booléennes :

```
type CheckNumber<T> = T extends number ? 'Number' : 'Not a number';
type A = CheckNumber<123>; // 'Number'
type B = CheckNumber<'abc'>; // 'Not a number'
```

La récursivité désigne une définition de type qui fait référence à elle-même au sein de sa propre définition :

```
type Json = string | number | boolean | null | Json[] | { [key: string]: Json };

const data: Json = {
    prop1: true,
    prop2: 'prop2',
    prop3: {
        prop4: [],
    },
};
```

Les types conditionnels récursifs combinent la logique conditionnelle et la récursivité. Cela signifie qu'une définition de type peut dépendre d'elle-même par l'intermédiaire d'une logique conditionnelle, créant ainsi des relations de types complexes et flexibles.

```
type Flatten<T> = T extends Array<infer U> ? Flatten<U> : T;

type NestedArray = [1, [2, [3, 4], 5], 6];
type FlattenedArray = Flatten<NestedArray>; // 2 | 3 | 4 | 5 | 1 | 6
```

Prise en charge des modules ECMAScript dans Node

Node.js a ajouté la prise en charge des modules ECMAScript à partir de la version 15.3.0, et TypeScript prend en charge les modules ECMAScript pour Node.js depuis la version 4.7. Cette prise en charge peut être activée en utilisant la propriété `module`

avec la valeur `nodenext` dans le fichier tsconfig.json. Voici un exemple :

```
{
  "compilerOptions": {
    "module": "nodenext",
    "outDir": "./lib",
    "declaration": true
  }
}
```

Node.js prend en charge deux extensions de fichier pour les modules : `.mjs` pour les modules ES et `.cjs` pour les modules CommonJS. Les extensions de fichier équivalentes dans TypeScript sont `.mts` pour les modules ES et `.cts` pour les modules CommonJS. Lorsque le compilateur TypeScript transpile ces fichiers en JavaScript, il crée des fichiers `.mjs` et `.cjs`.

Si vous souhaitez utiliser les modules ES dans votre projet, vous pouvez définir la propriété `type` sur “module” dans votre fichier package.json. Cela indique à Node.js de traiter le projet comme un projet de modules ES.

TypeScript prend également en charge les déclarations de types dans les fichiers .d.ts. Ces fichiers de déclaration fournissent des informations de type pour les bibliothèques ou modules écrits en TypeScript, permettant ainsi aux autres développeurs de les utiliser avec la vérification des types et les fonctionnalités d’autocomplétion de TypeScript.

Fonctions d’assertion

Dans TypeScript, les fonctions d'assertion sont des fonctions qui indiquent qu'une condition spécifique a été vérifiée en fonction de leur valeur de retour. Dans sa forme la plus simple, une fonction d'assertion examine un prédicat fourni et lève une erreur lorsque le prédicat est évalué à false.

```
function isNumber(value: unknown): asserts value is number {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
}
```

Elle peut également être déclarée sous forme d'expression de fonction :

```
type AssertIsNumber = (value: unknown) => asserts value is
number;
const isNumber: AssertIsNumber = value => {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
};
```

Les fonctions d'assertion présentent des similitudes avec les gardes de type. Les gardes de type ont été introduites initialement pour effectuer des vérifications à l'exécution et garantir le type d'une valeur dans une portée spécifique. Plus précisément, une garde de type est une fonction qui évalue un prédicat de type et renvoie une valeur booléenne indiquant si le prédicat est vrai ou faux. Cela diffère légèrement des fonctions d'assertion, dont l'objectif est de lever une erreur plutôt que de renvoyer false lorsque le prédicat n'est pas satisfait.

Exemple de garde de type :

```
const isNumber = (value: unknown): value is number => typeof
value === 'number';
```

Types de tuples variadiques

Les types de tuples variadiques sont une fonctionnalité introduite dans TypeScript 4.0. Commençons donc par revoir ce qu'est un tuple :

Un type tuple est un tableau dont la longueur est définie et dont le type de chaque élément est connu :

```
type Student = [string, number];
const [name, age]: Student = ['Simone', 20];
```

Le terme « variadique » désigne une arité indéfinie (l'acceptation d'un nombre variable d'arguments).

Un tuple variadique est un type tuple qui possède toutes les propriétés précédentes, mais dont la forme exacte n'est pas encore définie :

```
type Bar<T extends unknown[]> = [boolean, ...T, number];

type A = Bar<[boolean]>; // [boolean, boolean, number]
type B = Bar<['a', 'b']>; // [boolean, 'a', 'b', number]
type C = Bar<[]>; // [boolean, number]
```

Dans le code précédent, nous pouvons voir que la forme du tuple est définie par le paramètre de type générique `T` fourni.

Les tuples variadiques peuvent accepter plusieurs paramètres de type génériques, ce qui les rend très flexibles :

```
type Bar<T extends unknown[], G extends unknown[]> = [...T, boolean, ...G];

type A = Bar<[number], [string]>; // [number, boolean, string]
type B = Bar<['a', 'b'], [boolean]>; // ["a", "b", boolean, boolean]
```

Avec les nouveaux tuples variadiques, nous pouvons utiliser :

- Les opérateurs de décomposition dans la syntaxe des types tuples peuvent désormais être génériques. Nous pouvons ainsi représenter des opérations d'ordre supérieur sur des tuples et des tableaux, même lorsque nous ne connaissons pas les types réels sur lesquels nous opérons.
- Les éléments rest peuvent apparaître n'importe où dans un tuple.

Exemple :

```
type Items = readonly unknown[];

function concat<T extends Items, U extends Items>(
    arr1: T,
    arr2: U
): [...T, ...U] {
    return [...arr1, ...arr2];
}

concat([1, 2, 3], ['4', '5', '6']); // [1, 2, 3, "4", "5", "6"]
```

Types d'objets enveloppeurs

Les types d'objets enveloppeurs font référence aux objets enveloppeurs utilisés pour représenter les types primitifs sous forme d'objets. Ces objets enveloppeurs fournissent des

fonctionnalités et des méthodes supplémentaires qui ne sont pas directement disponibles sur les valeurs primitives.

Lorsque vous accédez à une méthode comme `charAt` ou `normalize` sur une primitive `string`, JavaScript l'enveloppe dans un objet `String`, appelle la méthode, puis supprime l'objet.

Démonstration :

```
const originalNormalize = String.prototype.normalize;
String.prototype.normalize = function () {
    console.log(this, typeof this);
    return originalNormalize.call(this);
};
console.log('\u0041'.normalize());
```

TypeScript représente cette distinction en fournissant des types distincts pour les primitives et les objets enveloppeurs correspondants :

- string => String
- number => Number
- boolean => Boolean
- symbol => Symbol
- bigint => BigInt

Les types d'objets enveloppeurs ne sont généralement pas nécessaires. Évitez de les utiliser et employez plutôt les types primitifs, par exemple `string` au lieu de `String`.

Covariance et contravariance dans TypeScript

La covariance et la contravariance décrivent les relations entre types dans les types génériques. Un `Dog` est un `Animal`, mais tous les `Animal` ne sont pas des `Dog`.

Covariance (production) : une fonction qui renvoie un `Dog` peut remplacer une fonction qui renvoie un `Animal`. Tout chien est un animal, mais une fonction renvoyant un animal quelconque pourrait renvoyer un chat.

```
class Animal {
    name = '';
}

class Dog extends Animal {
    breed = '';
}

type Producer<T> = () => T;

declare let produceAnimal: Producer<Animal>;
declare let produceDog: Producer<Dog>;

produceAnimal = produceDog; // Valide
produceDog = produceAnimal; // Erreur
```

Contravariance (consommation) : une fonction qui accepte tout `Animal` peut remplacer une fonction qui accepte un `Dog`. Elle accepte aussi les chiens ; une fonction réservée aux chiens ne peut pas traiter tous les animaux.

```
type Consumer<T> = (value: T) => void;

declare let consumeAnimal: Consumer<Animal>;
declare let consumeDog: Consumer<Dog>;

consumeDog = consumeAnimal; // Valide
consumeAnimal = consumeDog; // Erreur avec strictFunctionTypes
```

Les flèches indiquent le sens des affectations sûres :

- Dog -> Animal
- Producer<Dog> -> Producer<Animal> (covariance)
- Consumer<Animal> -> Consumer<Dog> (contravariance)

Tableaux modifiables : TypeScript permet aussi d'affecter `Dog[]` à `Animal[]`. Les variables désignent le même tableau ; insérer un `Animal` via le type plus général peut ajouter un élément qui n'est pas un chien.

```
const dogs: Dog[] = [new Dog()];
const animals: Animal[] = dogs;

animals.push(new Animal()); // Autorisé, mais non sûr
// dogs contient maintenant un Animal qui n'est pas un Dog.
```

Option du compilateur : avec `strictFunctionTypes`, les paramètres des fonctions indépendantes sont vérifiés de manière contravariante. Sans cette option, TypeScript peut accepter les deux sens (bivariance), même sans garantie de sécurité. Les paramètres déclarés dans les méthodes et constructeurs font exception.

Annotations de variance facultatives pour les paramètres de type

Depuis TypeScript 4.7, `out` indique la covariance, `in` la contravariance et `in out` l'invariance. TypeScript déduit normalement la variance. Les annotations doivent correspondre à l'utilisation du paramètre générique, sans la modifier arbitrairement.

```
type Producer<out T> = () => T;
type Consumer<in T> = (value: T) => void;
type Transformer<in out T> = (value: T) => T;
```

Signatures d'index avec motifs de chaînes de gabarit

Les signatures d'index avec motifs de chaînes de gabarit nous permettent de définir des signatures d'index flexibles à l'aide de motifs de chaînes de gabarit. Cette fonctionnalité permet de créer des objets qui peuvent être indexés avec des motifs spécifiques de clés de chaîne, offrant ainsi davantage de contrôle et de précision lors de l'accès aux propriétés et de leur manipulation.

Depuis la version 4.4, TypeScript autorise les signatures d'index pour les symboles et les motifs de chaînes de gabarit.

```
const uniqueSymbol = Symbol('description');

type MyKeys = `key-${string}`;

type MyObject = {
    [uniqueSymbol]: string;
    [key: MyKeys]: number;
};

const obj: MyObject = {
    [uniqueSymbol]: 'Unique symbol key',
    'key-a': 123,
    'key-b': 456,
};

console.log(obj[uniqueSymbol]); // Clé de symbole unique
console.log(obj['key-a']); // 123
console.log(obj['key-b']); // 456
```

L'opérateur satisfies

L'opérateur `satisfies` vous permet de vérifier si un type donné satisfait une interface ou une condition particulière. Autrement

dit, il garantit qu'un type possède toutes les propriétés et méthodes requises par une interface particulière. C'est un moyen de s'assurer qu'une variable correspond à la définition d'un type. Voici un exemple :

```ts
type Columns = 'name' | 'nickName' | 'attributes';

type User = Record<Columns, string | string[] | undefined>;

// Annotation de type utilisant `User`
const user: User = {
    name: 'Simone',
    nickName: undefined,
    attributes: ['dev', 'admin'],
};

// Dans les lignes suivantes, TypeScript ne pourra pas inférer correctement les types
user.attributes?.map(console.log);  // La propriété 'map' n'existe pas sur le type 'string | string[]'. La propriété 'map' n'existe pas sur le type 'string'.
user.nickName; // string | string[] | undefined

// Assertion de type utilisant `as`
const user2 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} as User;

// Ici aussi, TypeScript ne pourra pas inférer correctement les types
user2.attributes?.map(console.log);  // La propriété 'map' n'existe pas sur le type 'string | string[]'. La propriété 'map' n'existe pas sur le type 'string'.
user2.nickName; // string | string[] | undefined
```

```
// L'opérateur `satisfies` permet maintenant d'inférer correctement les types
const user3 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} satisfies User;

user3.attributes?.map(console.log);    // TypeScript infère correctement : string[]
user3.nickName; // TypeScript infère correctement : undefined
```

Importations et exportations de types uniquement

Les importations et exportations de types uniquement vous permettent d'importer ou d'exporter des types sans importer ni exporter les valeurs ou fonctions associées à ces types. Cela peut être utile pour réduire la taille de votre bundle.

Pour utiliser des importations de types uniquement, vous pouvez employer le mot-clé `import type`.

TypeScript autorise l'utilisation des extensions de fichiers de déclaration et d'implémentation (.ts, .mts, .cts et .tsx) dans les importations de types uniquement, quel que soit le réglage de `allowImportingTsExtensions`.

Par exemple :

```
import type { House } from './house.ts';
```

Les formes suivantes sont prises en charge :

```
import type T from './mod';
import type { A, B } from './mod';
import type * as Types from './mod';
```

```ts
export type { T };
export type { T } from './mod';
```

Déclaration using et gestion explicite des ressources

Une déclaration `using` est une liaison immuable à portée de bloc, similaire à `const`, utilisée pour gérer les ressources libérables. Lorsqu'elle est initialisée avec une valeur, la méthode `Symbol.dispose` de cette valeur est enregistrée puis exécutée lors de la sortie de la portée du bloc englobant.

Cette fonctionnalité repose sur la gestion des ressources d'ECMAScript, qui est utile pour effectuer les tâches de nettoyage essentielles après la création d'un objet, comme fermer des connexions, supprimer des fichiers et libérer de la mémoire.

Remarques :

- En raison de son introduction récente dans la version 5.2 de TypeScript, la plupart des environnements d'exécution ne la prennent pas en charge nativement. Vous aurez besoin de polyfills pour : `Symbol.dispose`, `Symbol.asyncDispose`, `DisposableStack`, `AsyncDisposableStack`, `SuppressedError`.
- En outre, vous devrez configurer votre tsconfig.json comme suit :

```
{
    "compilerOptions": {
        "target": "es2022",
        "lib": ["es2022", "esnext.disposable", "dom"]
    }
}
```

Exemple :

```
//@ts-ignore
Symbol.dispose ??= Symbol('Symbol.dispose'); // Polyfill simple

const doWork = (): Disposable => {
    return {
        [Symbol.dispose]: () => {
            console.log('disposed');
        },
    };
};

console.log(1);

{
    using work = doWork(); // La ressource est déclarée
    console.log(2);
}    //    La    ressource    est    libérée    (par    exemple,
`work[Symbol.dispose]()` est évalué)

console.log(3);
```

Le code affichera :

```
1
2
disposed
3
```

Une ressource pouvant être libérée doit respecter l'interface `Disposable` :

```
// lib.esnext.disposable.d.ts
interface Disposable {
    [Symbol.dispose](): void;
}
```

Les déclarations `using` enregistrent les opérations de libération des ressources dans une pile, garantissant qu'elles sont libérées

dans l'ordre inverse de leur déclaration :

```
{
    using j = getA(),
        y = getB();
    using k = getC();
} // libère `C`, puis `B`, puis `A`.
```

Il est garanti que les ressources seront libérées, même si du code ultérieur est exécuté ou si des exceptions surviennent. L'opération de libération peut ainsi lever une exception et éventuellement en masquer une autre. Afin de conserver les informations sur les erreurs masquées, une nouvelle exception native, `SuppressedError`, est introduite.

Déclaration await using

Une déclaration `await using` gère une ressource pouvant être libérée de manière asynchrone. La valeur doit posséder une méthode `Symbol.asyncDispose`, dont le résultat sera attendu à la fin du bloc.

```
async function doWorkAsync() {
      await using work = doWorkAsync(); // La ressource est déclarée
}   //  La  ressource  est  libérée  (par  exemple,  `await work[Symbol.asyncDispose]()` est évalué)
```

Pour une ressource pouvant être libérée de manière asynchrone, celle-ci doit respecter l'une des interfaces `Disposable` ou `AsyncDisposable` :

```
// lib.esnext.disposable.d.ts
interface AsyncDisposable {
    [Symbol.asyncDispose](): Promise<void>;
}
```

```
//@ts-ignore
Symbol.asyncDispose   ??=   Symbol('Symbol.asyncDispose');   //
Polyfill simple

class DatabaseConnection implements AsyncDisposable {
     // Méthode appelée lorsque l'objet est libéré de manière
asynchrone
    [Symbol.asyncDispose]() {
        // Fermer la connexion et renvoyer une promesse
        return this.close();
    }

    async close() {
        console.log('Closing the connection...');
             await new Promise(resolve => setTimeout(resolve,
1000));
        console.log('Connection closed.');
    }
}

async function doWork() {
     // Créer une connexion et la libérer de manière asynchrone
lorsqu'elle sort de la portée
     await using connection = new DatabaseConnection(); //  La
ressource est déclarée
    console.log('Doing some work...');
}   //   La   ressource   est   libérée   (par   exemple,   `await
connection[Symbol.asyncDispose]()` est évalué)

doWork();
```

Le code affiche :

```
Doing some work...
Closing the connection...
Connection closed.
```

Les déclarations `using` et `await using` sont autorisées dans les instructions : `for`, `for-in`, `for-of`, `for-await-of`, `switch`.

Attributs d'importation

Les attributs d'importation de TypeScript 5.3 (étiquettes pour les importations) indiquent à l'environnement d'exécution comment traiter les modules (JSON, etc.). Cela améliore la sécurité en garantissant des importations explicites et s'aligne sur la Content Security Policy (CSP) afin de rendre le chargement des ressources plus sûr. TypeScript vérifie leur validité, mais laisse l'environnement d'exécution les interpréter pour le traitement spécifique des modules.

Exemple :

```
import config from './config.json' with { type: 'json' };
```

avec une importation dynamique :

```
const config = import('./config.json', { with: { type: 'json' }
});
```

Vérification de la syntaxe des expressions régulières

Depuis TypeScript 5.5.4, les littéraux d'expression régulière sont vérifiés à la compilation afin de détecter les erreurs courantes (par exemple, une syntaxe non valide, des références arrière incorrectes ou des fonctionnalités non prises en charge par la version JavaScript cible). Cela permet de détecter les bogues plus tôt, mais les chaînes passées à new RegExp("...") ne sont pas vérifiées.

```
let r = /(a)\2/; // Erreur : cette référence arrière désigne un
groupe qui n'existe pas.
```

import defer

`import defer` vous permet de charger un module tout en retardant son exécution jusqu'à ce que vous utilisiez effectivement l'un de ses éléments. Cela permet d'éviter le travail et les effets secondaires inutiles.

- Fonctionne uniquement avec : `import defer * as name from "module"`
- Le code ne s'exécute que lorsque vous accédez à une exportation